TRABAJO FIN DE GRADO

GRADO EN INGENIERÍA INFORMÁTICA

Sistema de tutorización inteligente para la enseñanza de programación a niños

Diseño e implementación de una plataforma web con un sistema de tutorización inteligente para facilitar la enseñanza

Autor
Noelia Carrasco Vilar

Director
José Manuel Benítez Sánchez

Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación

—

Granada, Noviembre de 2023

Sistema de tutorización inteligente para la enseñanza de programación a niños

Noelia Carrasco Vilar

Palabras clave: *Sistema de Tutorización Inteligente*, *Educación Infantil*, *Análisis*, *Automatización*, *Programación*

Resumen

La enseñanza de la programación a edades tempranas se ha vuelto fundamental en la sociedad actual. Es en este contexto donde los Sistemas de Tutorización Inteligente (ITS) emergen como herramientas potenciales para apoyar un aprendizaje más personalizado y adaptado a las necesidades individuales de cada estudiante.

Este trabajo presenta el diseño, desarrollo y evaluación de un ITS enfocado en la enseñanza de programación. El sistema propuesto aplica una automatización en la corrección de los ejercicios con tal de poder ofrecer una buena retroalimentación en tiempo real. Además de adaptarse a las necesidades individuales de los estudiantes y monitorizar su progreso.

El proceso de desarrollo del ITS se llevó a cabo en diversas fases. En primer lugar, se establecieron los fundamentos pedagógicos y se identificaron las necesidades específicas del público infantil. Posteriormente, se integraron técnicas avanzadas de corrección y elección de ejercicios con tal de garantizar la adaptabilidad y efectividad del sistema.

Para evaluar la eficacia, se realizó una prueba con un pequeño grupo de estudiantes, en colaboración con la empresa CODELEARN S.L. Los resultados iniciales sugieren una posible mejora significativa en la retención de conocimientos y en el rendimiento académico de los participantes, validando así la propuesta de este TFG.

En conclusión, este Trabajo de Fin de Grado aporta una solución innovadora al campo de la educación tecnológica infantil, demostrando la viabilidad y eficacia de los Sistemas de Tutorización Inteligente dentro del mundo educativo de la programación.

Intelligent Tutoring System for Teaching Programming to Children

Noelia Carrasco Vilar

Keywords: *Intelligent Tutoring System*, *Early Childhood Education*, *Analysis*, *Automation*, *Programming*

Abstract

Teaching programming at an early age has become essential in today's society. It is in this context that Intelligent Tutoring Systems (ITS) emerge as potential tools to support more personalized learning tailored to the individual needs of each student.

This work presents the design, development, and evaluation of an ITS focused on teaching programming to children. The proposed system applies automation techniques in exercise correction to provide real-time feedback on the student's understanding. It is also capable of adapting to individual student needs and monitoring their progress.

The development of the ITS was carried out in various phases. Firstly, pedagogical foundations were established, and the specific needs of the child audience were identified. Advanced correction and exercise selection techniques were then integrated to ensure the adaptability and effectiveness of the system.

To evaluate the effectiveness of the ITS, tests and experiments were conducted with a small group , in collaboration with the company CODELEARN S.L. Initial results suggest a possible significant improvement in knowledge retention and academic performance of the participants, thus validating the proposal of this Bachelor's Thesis.

In conclusion, this Bachelor's Thesis provides an innovative solution to the field of early childhood technological education, demonstrating the feasibility and effectiveness of Intelligent Tutoring Systems in the educational programming world.

Yo, **Noelia Carrasco Vilar**, alumna de la titulación Grado en Ingeniería Informática de la **Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación de la Universidad de Granada**, con DNI 39475493H, autorizo la ubicación de la siguiente copia de mi Trabajo Fin de Grado en la biblioteca del centro para que pueda ser consultada por las personas que lo deseen.

Fdo: Noelia Carrasco Vilar

Granada a 13 de Noviembre de 2023.

D. **José Manuel Benítez Sánchez**, catédratico del departamento de Ciencias de la Computación e Inteligencia Artificial de la Universidad de Granada.

Informa:

Que el presente trabajo, titulado ***Sistema de tutorización inteligente para la enseñanza de programación a niños***, ha sido realizado bajo su supervisión por **Noelia Carrasco Vilar**, y autoriza la defensa de dicho trabajo ante el tribunal que corresponda.

Y para que conste, expide y firma el presente informe en Granada a 13 de Noviembre de 2023.

El director:

José Manuel Benítez Sánchezz

Agradecimientos

Quiero comenzar agradeciendo a los profesores que han realizado un compromiso para la formación académica de mi promoción. Su pasión y dedicación fomentaron mi interés por la informática, lo cual me ha llevado a realizar este TFG. Un agradecimiento especial a mi tutor del TFG, quien guio y ayudó este proyecto.

Agradezco de manera especial a la empresa Codelearn S.L. por su apoyo y colaboración en este proyecto. Su experiencia y recursos han sido cruciales para el desarrollo de mi TFG.

No puedo pasar por alto el papel fundamental que mi familia ha desempeñado en mi vida, tanto académica como personal. A mis padres y a mi hermano, cuya influencia ha sido determinante en mi formación. Dedico este trabajo y los proyectos futuros, como reconocimiento a todos los esfuerzos que han realizado para permitirme alcanzar mis metas académicas. También deseo honrar la memoria de mis abuelos, quienes, presentes o ausentes, han sido una fuente de inspiración constante a lo largo de mi trayectoria.

Índice general

1. Introducción **1**
1.1. Contexto Actual y Relevancia de la Programación 1
1.2. Descripción del Problema 1
1.3. Enfoque 2
1.3.1. Objetivos del Proyecto 2
1.3.2. Desarrollo Ágil 2
1.4. Colaboración con Codelearn S.L. 3
1.5. Estructura de la memoria 3

2. Estado del Arte y Trabajos Relacionados **5**
2.1. Definición de los ITS 5
2.2. Evolución de los ITS 6
2.3. Aplicaciones actuales de los ITS 7
2.3.1. Educación K-12 7
2.3.2. Educación Superior 7
2.3.3. Formación y Capacitación Corporativa 8
2.3.4. Salud y Medicina 8
2.3.5. Defensa y Aviación 8
2.3.6. Juegos Educativos y Simuladores 8
2.4. Limitaciones y Desafíos 9
2.5. Papel del ITS en la educación moderna 9
2.5.1. Metodología para la adaptación del aprendizaje 10
2.5.2. Futuras Direcciones 10
2.6. Comparativa de las plataformas de programación 12

3. Planificación **13**
3.1. Fases 13
3.1.1. Fase Inicial 13
3.1.2. Fase de análisis 13
3.1.3. Fase de desarrollo 14
3.1.4. Fase de prueba y corrección 15
3.1.5. Elaboración de la memoria 15
3.2. Presupuesto 16

3.2.1. Licencias 16
3.2.2. Recursos Materiales 17
3.2.3. Costes de Personal 17
3.2.4. Otros costes 17
3.2.5. Total 18

4. Análisis **19**
4.1. Análisis de Usuarios Objetivo 19
4.2. Análisis de Requisitos Educativos 20
4.3. Modelos de Caso de Uso y Escenarios de Uso 20
4.3.1. Actores del Sistema 21
4.3.2. Escenarios de Uso Detallados 22
4.3.3. Restricciones y Reglas de Negocio 26
4.3.4. Seguridad y Acceso 27
4.4. Modelado de Datos y Diagrama Entidad-Relación 28

5. Requisitos **31**
5.1. Requisitos Funcionales 31
5.1.1. Generales 32
5.1.2. Sistema 33
5.1.3. Estudiantes 34
5.1.4. Profesores 36
5.1.5. Administrador 38
5.2. Requisitos No Funcionales 40
5.3. Requisitos de Información 42

6. Diseño e Implementación **49**
6.1. Diseño Pedagógico 49
6.2. Arquitectura del Sistema 50
6.3. Diseño de Base de Datos 51
6.4. Interfaz de Usuario 53
6.4.1. *Mockups* de la página web 53
6.4.2. Interacción Usuario-Sistema 58
6.5. Implementación de Algoritmos 59
6.5.1. Corrección y evaluación de calidad del código 59
6.5.2. Selección y evaluación de ejercicios 60
6.5.3. Cálculo de la puntuación 61
6.5.4. Comprobaciones del código 62
6.5.5. Cálculo del número extra de ejercicios 64
6.6. Implementación de Módulos de Aprendizaje Adaptativo . . . 66

7. Despliegue **69**
7.1. Creación y Configuración de las Imágenes 69
7.2. Utilización de Podman 69
7.2.1. Contenedor flask_TFG 70
7.2.2. Contenedor db 70

8. Pruebas y Resultados **71**

9. Conclusiones y Trabajo Futuro **73**
9.1. Conclusiones 73
9.2. Trabajo Futuro 73
9.3. Reflexión Personal 74

Anexos **75**

A. Recursos Informáticos **75**
A.1. Hardware 75
A.2. Software 75
A.3. Recursos de Red y Servidores 76

B. Guion de las pruebas con niños **77**
B.1. Preparación 77
B.2. Participantes 77
B.3. Actividades 77
B.4. Feedback y Conclusiones 78

C. Ficheros usados para el despliegue **79**

D. Manual de usuario **83**
D.1. Captura de EScritorio 83
D.2. Capturas del Profesor 83
D.3. Capturas del Administrador 83

Glosario **85**

Bibliografía **87**

Bibliografía **91**

Índice de figuras

2.1. Arquitectura básica de un ITS [4] 6

3.1. Diagrama de Gantt con la planificación del TFG 16

4.1. Diagrama de Caso de Uso del sistema 22
4.2. Diagrama Entidad-Relación del sistema 29

6.1. Arquitectura de sistema 50
6.2. Diagrama del modelo de datos 52
6.3. Inicio de Sesión para cualquier usuario. 53
6.4. Registro para cualquier usuario. 53
6.5. Página Principal de la interfaz del estudiante. 54
6.6. Apartado de Juegos de la interfaz del alumno. 54
6.7. Visualización de la teoría para el estudiante. 54
6.8. Apartado de contacto con el profesor. 55
6.9. Apartado de Ranking de la interfaz del usuario. 55
6.10. Visualización de los ejercicios para el usuario. 55
6.11. Apartado donde los estudiantes pueden revisar conceptos teóricos. 56
6.12. Apartado de configuración del estudiante. 56
6.13. Dashboard principal del profesor. 56
6.14. Pantalla de Estadísticas del profesor. 57
6.15. Pantalla con la lista de corrección de ejercicios. 57
6.16. Panel para facilitar la corrección los ejercicios para el profesor. 57
6.17. Apartado de Configuración del profesor. 58
6.18. Visión general del panel administrativo. 58
6.19. Diagrama de flujo para la selección de ejercicios 60
6.20. Diagrama de flujo para la selección de ejercicios 61
6.21. Diagrama de flujo que ilustra cómo se calcula la puntuación . 62
6.22. Diagrama de flujo que ilustra cómo se calcula la puntuación . 63
6.23. Diagrama de flujo que ilustra el cálculo del número extra de ejercicios 65

Índice de tablas

2.1. Comparativa de plataformas actuales para la enseñanza de programación 12

3.1. Costes asociados al personal para un ingeniero informático. . 17
3.2. Total de costes del proyecto 18

4.1. EU-1: Realización de un ejercicio 23
4.2. EU-2: Tiempo superado en un ejercicio 24
4.3. EU-3: Corrección de ejercicios por parte del profesor 24
4.4. EU-4: Gestión del contenido didáctico 25
4.5. EU-5: Resolución de dudas 25
4.6. EU-6: Seguimiento del Progreso del Estudiante 25
4.7. EU-7: Gestión de usuarios 26

5.1. Requisito funcional RF-01: Inicio de sesión. 32
5.2. Requisito funcional RF-02: Registro de usuario. 32
5.3. Requisito funcional RF-03: Perfil de usuario. 32
5.4. Requisito funcional RF-04: Selección de ejercicios por requisitos. 33
5.5. Requisito funcional RF-05: Corrección de ejercicios. 33
5.6. Requisito funcional RF-06: Compilador integrado. 33
5.7. Requisito funcional RF-07: Categorización de la teoría. 34
5.8. Requisito funcional RF-08: Repetición de ejercicios similares tras fallo. 34
5.9. Requisito funcional RF-09: Límite de tiempo para completar un ejercicio. 34
5.10. Requisito funcional RF-10: Resolución de dudas. 34
5.11. Requisito funcional RF-11: Consulta de los rankings. 35
5.12. Requisito funcional RF-12: Juegos en la web. 35
5.13. Requisito funcional RF-13: Consulta de los ejercicios corregidos. 35
5.14. Requisito funcional RF-14: Sistema de notificación. 35
5.15. Requisito funcional RF-15: Feedback sobre la solución. 36
5.16. Requisito funcional RF-16: Sistema de puntos. 36
5.17. Requisito funcional RF-17: Lista de usuarios. 36
5.18. Requisito funcional RF-18: Consulta de ejercicios realizados. . 36

5.19. Requisito funcional RF-19: Panel de resolución de preguntas. 37
5.20. Requisito funcional RF-20: Lista de estudiantes inactivos. . . 37
5.21. Requisito funcional RF-21: Tiempo de resolución de ejercicios. 37
5.22. Requisito funcional RF-22: Corrección de ejercicios. 37
5.23. Requisito funcional RF-23: Usuarios con múltiples fallos. . . . 38
5.24. Requisito funcional RF-24: Ejercicios realizados por día. . . . 38
5.25. Requisito funcional RF-25: Usuarios con alta tasa de errores. 38
5.26. Requisito funcional RF-26: Alta de profesor por administrador. 38
5.27. Requisito funcional RF-27: Añadir elementos. 39
5.28. Requisito funcional RF-28: Consultar elementos. 39
5.29. Requisito funcional RF-29: Editar elementos. 39
5.30. Requisito funcional RF-30: Eliminar elementos. 39
5.31. Requisito funcional RF-31: Orden global. 40
5.32. Requisito no funcional RNF-01: Usabilidad. 40
5.33. Requisito no funcional RNF-02: Rendimiento. 40
5.34. Requisito no funcional RNF-03: Seguridad. 41
5.35. Requisito no funcional RNF-04: Compatibilidad. 41
5.36. Requisito no funcional RNF-05: Escalabilidad. 41
5.37. Requisito no funcional RNF-06: Disponibilidad. 41
5.38. Requisito no funcional RNF-07: Tiempo de Respuesta. 42
5.39. Requisito no funcional RNF-08: Mantenimiento. 42
5.40. Requisito de información RI-01: Usuario. 43
5.41. Requisito de información RI-02: Teoría. 43
5.42. Requisito de información RI-03: Ejercicios. 43
5.43. Requisito de información RI-04: Juegos. 44
5.44. Requisito de información RI-05: Progreso del estudiante. . . . 44
5.45. Requisito de información RI-06: Actividad del estudiante. . . 44
5.46. Requisito de información RI-07: Transacciones de Pagos. . . . 45
5.47. Requisito de información RI-08: Notificaciones. 45
5.48. Requisito de información RI-09: Requisitos. 45
5.49. Requisito de información RI-10: *Path* educativo. 46
5.50. Requisito de información RI-11: Módulo. 46
5.51. Requisito de información RI-12: Preguntas. 46
5.52. Requisito de información RI-13: Requisitos completados por el usuario. 47
5.53. Requisito de información RI-14: Requisitos de teoría. 47
5.54. Requisito de información RI-15: Requisitos de los ejercicios. . 47

Capítulo 1

Introducción

1.1. Contexto Actual y Relevancia de la Programación

En la era actual, la digitalización está en todos los aspectos de nuestra vida. Afecta desde cómo nos comunicamos hasta como realizamos nuestras tareas cotidianas. Dentro de este contexto, la programación ha transcendido su función original como mera herramienta técnica para convertirse en una habilidad esencial del siglo XXI. No solo es importante para aquellos que buscan una carrera tecnológica, sino también para el público que desea entender y utilizar eficazmente las herramientas digitales [44]. Sin embargo, la enseñanza de programación a niños representa desafíos únicos, no solo debido a la complejidad del contenido, sino también por las necesidades pedagógicas específicas que conlleva. [39].

1.2. Descripción del Problema

Tal como hemos hablado en la sección anterior, enseñar programación a niños es un desafío que no podemos ignorar. Existen diversas plataformas educativas, pero muchas tienen bastantes limitaciones. Ofrecen ejercicios y evaluaciones estáticas, sin tener en cuenta una adaptación al ritmo o habilidades del estudiante. Además, la mayoría, únicamente se centran en la corrección binaria del código. Por ello, ignoran aspectos como la eficiencia del mismo y el uso de buenas prácticas de programación. Este enfoque limitado no solo afecta a los estudiantes. También deja a profesores y administradores sin las herramientas necesarias para interactuar de una forma efectiva con el sistema. Es por eso que hay una necesidad clara e importante de una solución más completa y adaptativa para la enseñanza de programación a niños.

1.3. Enfoque

Este TFG adopta un enfoque multidisciplinario, combinando elementos de pedagogía, tecnología educativa y diseño de sistemas. La naturaleza del trabajo exige la interacción de varias disciplinas con tal de abordar de manera eficaz los desafíos presentados por la enseñanza de programación a niños.

1.3.1. Objetivos del Proyecto

El propósito principal de este TFG es desarrollar un enfoque integral para la enseñanza de programación a niños, teniendo en cuenta las particularidades y desafíos que este proceso conlleva. Los objetivos específicos se detallan a continuación:

1. **Identificación y evaluación de desafíos pedagógicos:** Analizar los retos específicos asociados con la enseñanza de programación a niños, incluyendo las necesidades pedagógicas y la complejidad del contenido.

2. **Diseño del Sistema de Tutorización Inteligente (ITS):** Crear una estructura para un ITS que sea adaptativo y personalizado, atendiendo a las necesidades individuales de aprendizaje de cada niño.

3. **Desarrollo de una metodología adaptativa para el ITS:** Formular una metodología que permita al ITS ajustarse basándose en el progreso y las respuestas de los estudiantes, mejorando así la eficacia del proceso de aprendizaje.

4. **Integración y optimización del ITS en una plataforma web:** Implementar el ITS en una plataforma web amigable y accesible, con un enfoque en la experiencia del usuario, asegurando que sea intuitiva y efectiva para niños y educadores.

5. **Experimentación y análisis práctico:** Probar el sistema en contextos educativos reales, recopilando datos y feedback para evaluar su eficacia y realizar los ajustes necesarios.

1.3.2. Desarrollo Ágil

Para abordar la dinámica cambiante y las necesidades de adaptabilidad del proyecto, se ha optado por usar un enfoque de desarrollo ágil para la implementación del software. Esto, favorece un proceso iterativo e incremental, permitiendo varios ajustes rápidos ante cualquier cambio. Además, de ofrecer la posibilidad de realizar entregas parciales que se pueden evaluar y ajustar en fases iniciales.

Esta estrategia de desarrollo asegura flexibilidad, fomenta la colaboración y se centra en el usuario final. Garantizando, así, que las soluciones

implementadas se alinean con las necesidades del ámbito de la enseñanza y el aprendizaje.

1.4. Colaboración con Codelearn S.L.

Este proyecto no habría sido posible sin la colaboración estrecha con Codelearn S.L. [5], una destacada entidad de la educación tecnológica para niños. A pesar de los avances y logros de Codelearn S.L., se identificó la necesidad de evolucionar y adaptar su sistema pedagógico para hacerlo más personalizado y efectivo. Esta colaboración ha permitido fusionar la experiencia y conocimientos de ambas partes, con el objetivo de desarrollar un ITS que permita mejorar la forma en que se enseña.

1.5. Estructura de la memoria

En esta sección se describe la estructura de esta memoria. Concretamente, se organiza en nueve capítulos principales, acompañados de anexos, que detallan el desarrollo.

1. **Introducción** 1: Establece el contexto de la programación en la era actual, describe el problema que aborda el TFG, presenta los objetivos, el enfoque metodológico, la colaboración con Codelearn S.L., y esta misma sección que esboza la estructura del documento.

2. **Estado del Arte y Trabajos Relacionados** 2: Profundiza en los Sistemas de Tutorización Inteligente (ITS), su evolución, aplicaciones actuales en diversos campos, limitaciones, desafíos y su papel en la educación moderna, incluyendo una comparativa de plataformas de programación.

3. **Planificación** 3: Desglosa las fases del proyecto junto con el presupuesto detallado que cubre licencias, recursos materiales, costes de personal y otros gastos.

4. **Análisis** 4: Incluye el análisis de usuarios objetivo, requisitos educativos, modelos de casos de uso, escenarios de uso, y la seguridad y acceso al sistema.

5. **Requisitos** 5: Define los requisitos funcionales, no funcionales y de información del sistema, detallando aspectos generales, del sistema, de estudiantes, profesores y administradores.

6. **Diseño e Implementación** 6: Cubre el diseño pedagógico, la arquitectura del sistema, diseño de base de datos, interfaz de usuario y la implementación de algoritmos y módulos de aprendizaje adaptativo.

7. **Despliegue** 7: Describe la creación y configuración de imágenes para el despliegue del sistema y el uso de Podman.

8. **Pruebas y Resultados** 8: Presenta las pruebas realizadas y los resultados obtenidos a partir de estas.

9. **Conclusiones y Trabajo Futuro** 9: Ofrece conclusiones derivadas del trabajo realizado, posibles direcciones futuras para la investigación y una reflexión personal sobre el proyecto.

Además, los anexos proporcionan información adicional sobre recursos informáticos A, guiones de pruebas con los niños B, ficheros utilizados para el despliegue C, un manual de usuario D, un glosario y la bibliografía.

Capítulo 2

Estado del Arte y Trabajos Relacionados

Dentro de este capítulo se va a presentar todo el conocimiento teórico adquirido para la preparación de la implementación y adaptación del ITS. Concreatamente, el estudio se ha basado en las contribuciones presentadas en las conferencias de la *International Conference on Intelligent Tutoring Systems* de los años 2016 [29], 2020 [30] y 2021 [31]. Estas ediciones han sido una fuente valiosa de información sobre las últimas tendencias, técnicas y avances en el campo de la tutoría inteligente.

2.1. Definición de los ITS

Los Sistemas de Tutoría Inteligente (ITS, por sus siglas en inglés) son sistemas computacionales diseñados que proporcionan instrucciones o una retroalimentación inmediata y personalizada a los estudiantes con la mínima o nula intervención humana [64]. Estos utilizan técnicas de Inteligencia Artificial para modelar el conocimiento del dominio que se enseña y el proceso de aprendizaje del estudiante [3]. A través de estos modelos, los ITS adaptan la presentación del material educativo y los métodos de instrucción a las necesidades individuales de cada estudiante [43].

La principal meta de un ITS es imitar el efecto beneficioso que tiene un tutor humano en el aprendizaje [61]. Para ello, los ITS abarcan diferentes componentes 2.1 como el modelo del estudiante, el modelo del tutor, el modelo del dominio y el modelo de interfaz [33]. Cada uno de estos modelos trabaja en conjunto para adaptar y personalizar el contenido y la retroalimentación, buscando optimizar el rendimiento académico y el compromiso del estudiante [16].

Una de las ventajas de los ITS es su habilidad para ofrecer una enseñanza adaptativa y escalable [42]. En contextos educativos donde la enseñanza uno a uno no es práctica o factible, los ITS representan una solución viable para

ofrecer instrucción personalizada a un gran número de estudiantes [25].

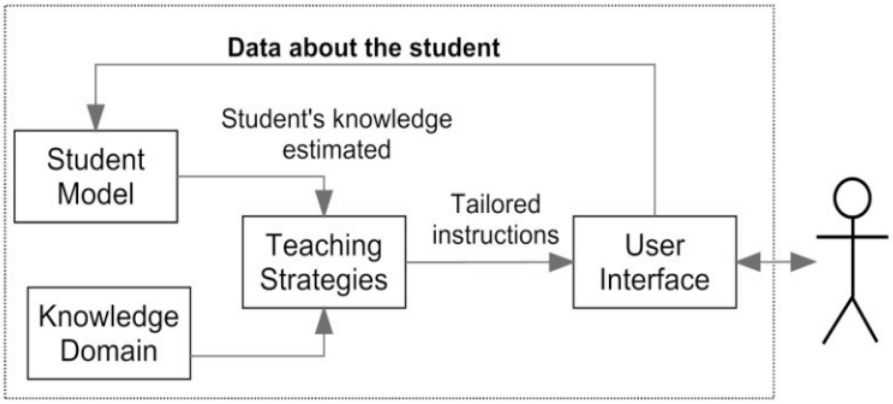

Figura 2.1: Arquitectura básica de un ITS [4]

2.2. Evolución de los ITS

La idea de ser capaces de utilizar la tecnología para personalizar y mejorar la experiencia de aprendizaje no es nueva. Sin embargo, con el crecimiento exponencial de la tecnología, la idea ha ido experimentando transformaciones radicales.

Los ITS nacieron en la década de 1970, con la idea de que la tecnología podía ser utilizada para simular la tutoría humana. Estos primeros sistemas, limitados por la capacidad computacional de entonces, se centraban en unos conocimientos muy específicos, como podían ser la geometría o el álgebra. Utilizaban una lógica básica y simple para ofrecer retroalimentación y una guía basada en las respuestas de los estudiantes. A pesar de su simplicidad, fueron representación de un gran paso hacia la individualización del aprendizaje.

Con el progreso en la investigación de la Inteligencia Artificial (IA) en la década de 1980, se comenzó a incorporar técnicas más avanzadas. Estos sistemas no solo eran capaces de evaluar las respuestas de los estudiantes, sino también de comprender parcialmente el proceso de pensamiento detrás de esas respuestas. Así, el feedback proporcionado se volvía más personalizado y adaptativo, reflejando mejor las necesidades individuales de cada alumno.

La siguiente década trajo consigo una mayor comprensión de la importancia de la interacción humano-computadora en los procesos educativos. Donde se desarrollaron unas interfaces más intuitivas y amigables para el usuario. Estos sistemas comenzaron a ofrecer unos mejores entornos de aprendizaje, incorporando multimedia, simulaciones y otros recursos interactivos. Con la idea de crear un entorno donde el estudiante no solo recibiera retroalimen-

tación, sino que también pudiera explorar y aprender de una manera más autónoma.

El siglo XXI trajo la proliferación de *Big Data* y el avance en técnicas de aprendizaje profundo. Esto permitió un nuevo nivel para los ITS. Con la capacidad de procesar grandes cantidades de datos, estos sistemas detectan patrones y tendencias dentro del comportamiento del estudiante que antes pasaban por desapercibidas. Usando toda esta información se predice las necesidades de aprendizaje futuras y se adapta el contenido en tiempo real, proporcionando una experiencia de aprendizaje aún más personalizada.

El potencial de los ITS sigue siendo muy amplio. Con la constante evolución de la tecnología y una comprensión cada vez más profunda de la pedagogía, es probable que veamos sistemas aún más avanzados en el futuro cercano. Incluso, se espera que la integración de realidades virtual y aumentada, junto con avances en IA, genere ITS que ofrezcan experiencias de aprendizaje inmersivas y altamente adaptativas.

2.3. Aplicaciones actuales de los ITS

La relevancia actual de los ITS se extiende a una variedad de dominios. Aprovechando técnicas de aprendizaje automático, estos sistemas pueden reconocer patrones en el comportamiento del usuario y adaptarse en función a ello. A su vez, mediante el análisis semántico, comprenden de manera profunda las respuestas de los estudiantes, garantizando una retroalimentación más precisa y relevante. Además, algunos ITS modernos están incorporando Realidad Virtual y Aumentada, ofreciendo entornos de aprendizaje inmersivos que permiten interacciones novedosas y atractivas con el contenido. A continuación, exploraremos algunas de las aplicaciones más prominentes de estos sistemas en el contexto actual.

2.3.1. Educación K-12

Los ITS se han establecido como herramientas fundamentales para complementar el aprendizaje tradicional dentro del ámbito escolar, ya sea en la educación primaria o en la eduación secundaria. Sistemas como *Dreambox* [18] para matemáticas o *Fast Forword* [38] para la lectura, utilizan técnicas de tutorización inteligente que adaptan el contenido a las necesidades individuales de cada estudiante, permitiendo una educación más personalizada.

2.3.2. Educación Superior

Las universidades y colegios alrededor del mundo están empezando a integrar ITS en sus plataformas de aprendizaje en línea. Sobre todo, estas instituciones los han comenzado a usar dentro de sus cursos masivos en línea

(MOOCs), donde el sistema se encarga de tutorizar a miles de estudiantes simultáneamente, sin una presencia física importante del profesor.

Por ejemplo, el sistema *edX* [65], iniciado por Harvard y el MIT, ha integrado algoritmos de tutorización inteligente para ofrecer retroalimentación personalizada a los estudiantes en sus cursos MOOCs con más inscritos. Asimismo, la Universidad de Stanford, a través de la plataforma *Coursera* [34], también ha experimentado con herramientas de tutorización para mejorar la retención y comprensión de los estudiantes adscritos.

2.3.3. Formación y Capacitación Corporativa

El mundo empresarial también ha reconocido el valor estos sistemas para la formación continua de sus empleados. Las grandes corporaciones han comenzado a implementarlos dentro de sus programas de formación, permitiendo la identificación y corrección de carencias de habilidades en tiempo real.

Grandes corporaciones como Google y Microsoft ya han comenzado a implementarlo en sus programas de formación interna. Por ejemplo, el *Google's IT Support Certificate* [24], un programa desarrollado para formar especialistas de soporte en TI, utiliza los ITS para guiar a los estudiantes a través de una serie de labores técnicas, identificando áreas de mejora en tiempo real.

2.3.4. Salud y Medicina

En el campo médico, también están desempeñando un papel crucial en la formación de futuros profesionales. Concretamente, *Touch Surgery Simulator* [41] permite a los médicos practicar procedimientos quirúrgicos en un entorno virtual, y proporciona una retroalimentación detallada basada en el rendimiento y decisiones tomadas durante la simulación.

2.3.5. Defensa y Aviación

Es importante destacar que la aviación y la defensa han estado al frente en la adopción de simuladores con capacidades de tutorización. Un gran ejemplo es el *F-35 Lightning II Training System* [40] que ayuda a capacitar los pilotos de combate, proporcionando escenarios adaptativos basados en las habilidades y el progreso del piloto.

2.3.6. Juegos Educativos y Simuladores

Los juegos educativos, como *DragonBox* [15] que enseña matemáticas, o *Civilization EDU* [23] que instruye sobre historia y estrategia, también integran sistemas de tutorización inteligente para adaptar el contenido y los desafíos a las habilidades del jugador, asegurando una curva de aprendizaje apropiada.

2.4. Limitaciones y Desafíos

A pesar de las prometedoras capacidades y aplicaciones de los ITS en diversos contextos educativos y de formación, estos sistemas no están exentos de limitaciones y desafíos.

La interacción con estos sistemas no logra capturar la profundidad y empatía de un tutor humano, lo que genera una desconexión o frustración por parte del estudiante. Estos sistemas, siendo fruto de una confluencia de disciplinas muy variadas, plantean retos en su diseño y actualización, especialmente cuando se busca mantener su relevancia en campos en constante evolución.

La adaptabilidad de estos sistemas también se encuentra en una línea delicada, por ser lo suficientemente generales para diversos contextos, o demasiado específicos para brindar una retroalimentación precisa. Siendo la búsqueda de este equilibrio esencial para garantizar su eficacia. Cabe destacar, que con la recopilación constante de datos del usuario, se debe tener especial delicadeza en cuanto a la privacidad y seguridad. Siendo imperativo que ofrezcan unas garantías robustas con tal de proteger la información del alumno.

Por otro lado, a pesar de su potencial democratizador en el acceso a la educación, los ITS corren el riesgo de convertirse en herramientas elitistas, accesibles solo para aquellos con los medios para obtenerlos. Por ello, es crucial que se trabaje en su accesibilidad y asequibilidad. Y si bien estos sistemas ofrecen métricas de rendimiento, medir su impacto real en el aprendizaje y en la transferencia práctica del conocimiento sigue siendo un gran reto.

En resumen, se podría decir que los ITS se ven como una herramienta revolucionaria, pero es fundamental enfrentar y superar los desafíos mencionados para garantizar que su implementación sea efectiva y equitativa en todos los contextos educativos.

2.5. Papel del ITS en la educación moderna

Los ITS representan un cambio pragmático en el ámbito educativo. Utilizando algoritmos de inteligencia artificial, estos sistemas adaptan el contenido y el ritmo de aprendizaje de manera personalizada para cada estudiante [3]. La adaptabilidad es una de las características más destacadas de, permitiendo un ajuste en tiempo real del material educativo según el rendimiento y las necesidades del alumno [32].

Esta capacidad para personalizar la instrucción ha demostrado ser eficaz para mejorar tanto el rendimiento académico como la retención del material. Además, los ITS recopilan datos detallados sobre el proceso de aprendizaje, lo que permite un análisis profundo para la identificación de áreas problemáticas y la facilitación de intervenciones educativas más eficaces [61].

Aunque ofrecen grandes ventajas, también existen desafíos y limitaciones en su implementación. La efectividad de un ITS puede variar según el contexto y la materia, y su complejidad técnica puede ser una barrera para su adopción en entornos educativos más amplios [47].

2.5.1. Metodología para la adaptación del aprendizaje

La adaptabilidad del aprendizaje es esencial para maximizar el potencial de cada estudiante y garantizar que el material sea presentado de una manera tanto comprensible como desafiante. Una metodología eficaz para esta adaptabilidad debería incluir componentes como análisis de datos en tiempo real, una retroalimentación y personalización del contenido instantánea.

La recolección y el análisis de datos en tiempo real permiten al sistema entender mejor las necesidades y los puntos débiles del estudiante [19]. Esto a su vez facilita una retroalimentación más precisa y contextual, lo que mejora la eficacia del aprendizaje [27].

Una característica importante es la adaptación de contenido, que puede variar desde la velocidad de la presentación del material hasta la complejidad de los ejercicios. Los ITS son particularmente eficaces en este aspecto, al permitir una adaptabilidad en tiempo real personalizada para cada estudiante [3].

En conjunto, la metodología debe ser iterativa y flexible, capaz de adaptarse tanto a las tendencias a largo plazo como a las necesidades inmediatas del estudiante, para poder proporcionar una experiencia de aprendizaje optimizada.

2.5.2. Futuras Direcciones

A medida que la tecnología y la pedagogía avanzan, se abren nuevos horizontes para los ITS que prometen transformar aún más la experiencia educativa. En este apartado, se explorarán algunas de las direcciones futuras más prometedoras dentro del campo de la investigación y desarrollo de ITS.

Integración de Realidades Virtuales y Aumentadas

La incorporación de tecnologías de Realidad Virtual (RV) y Realidad Aumentada (RA) en los ITS tiene el potencial de crear entornos de aprendizaje inmersivos y altamente interactivos. Esto permitir´ia que los estudiantes se enganchen con el material de forma más profunda, facilitando tanto el aprendizaje conceptual como el práctico [48].

Adaptabilidad Multidisciplinaria

Los ITS actuales suelen centrarse en dominios específicos, como las matemáticas o la programación. La adaptabilidad multidisciplinaria se presenta

como un desafío futuro, donde los ITS serían capaces de guiar al estudiante a través de un currículo más amplio, ajustando dinámicamente los planes de estudio en función del progreso en múltiples materias [2].

Emociones y Aprendizaje

Entender y adaptarse a las emociones del estudiante representa una dimensión adicional de personalización. Investigaciones futuras deberían enfocarse en cómo los ITS pueden detectar y responder a las emociones del estudiante para mejorar el compromiso y la retención [17].

Equidad en el Acceso

La equidad en el acceso a los ITS eficaces para estudiantes de todos los entornos socioeconómicos es una preocupación creciente. Los esfuerzos futuros podrían centrarse en hacer que estos sistemas sean más accesibles y asequibles, para evitar la creación de una brecha educativa [49].

Ética y Privacidad

Dada la recopilación de datos del estudiante, la ética y la privacidad son de suma importancia. Los ITS futuros deberán implementar garantías robustas de privacidad, cumpliendo con regulaciones globales con tal de asegurarse la confianza del usuario [66].

Interacción Humano-Agente

La eficacia de la interacción humano-agente se espera que sea un área de enfoque en los ITS futuros. La investigación podría orientarse hacia el desarrollo de agentes de IA que puedan simular con mayor eficacia la empatía y el apoyo que un educador humano podría proporcionar [28].

Validación y Efectividad

Medir la efectividad de los ITS en términos de resultados de aprendizaje y transferencia de habilidades es crítico. Estudios longitudinales que comparen la eficacia de los ITS con los métodos tradicionales de enseñanza podrían ofrecer insights valiosos [8].

En resumen, los ITS tienen un camino fascinante por delante. Sin embargo, es crucial que la comunidad académica y la industria trabajen en conjunto para superar los desafíos actuales, permitiendo que los ITS alcancen su potencial completo en la mejora y democratización de la educación.

2.6. Comparativa de las plataformas de programación

Plataforma	**Edad**	**Enfoque**	**Coste**	**IT$**	**Funcionamiento**	**Lenguajes**
Codecademy	Adolescentes y adultos	Interactivo	De pago	Sí	Web	Python, Java
Codewars	Adolescentes y adultos	Desafíos	Gratuito	No	Web	Python, Ruby
edX	Adolescentes y adultos	Cursos universitarios	De pago y gratuito	Sí	Web	Python, Java
Coursera	Adolescentes y adultos	Variedad de cursos	De pago y gratuito	Sí	Web y App	Python, C++
JetBrains Academy	Adolescentes y adultos	Proyectos prácticos	De pago	No	Web	Java, Kotlin
Codelearn	Niños	Intro a programación y robótica	De pago	Sí	Web	Logo, Scratch, Python
Code.org	Niños y adolescentes	Amplia gama de actividades	Gratuito	Sí	Web	Blockly, JavaScript

Tabla 2.1: Comparativa de plataformas actuales para la enseñanza de programación

Capítulo 3

Planificación

La planificación es un aspecto importante en cualquier proyecto, debido a que permite asegurar una correcta ejecución y el logro de los objetivos establecidos. Por ello, en este capítulo, se aborda la organización y estructuración del proyecto con el fin de cumplir el objetivo principal. Además, se adjunta un diagrama de Gantt 3.1 para ofrecer una visión detallada del tiempo asignado a cada tarea.

3.1. Fases

3.1.1. Fase Inicial

En la fase inicial se establecieron las bases y objetivos del proyecto. Se reconoció la necesidad de una plataforma educativa digital que enseñase tanto programación como sus buenas prácticas. Se llevaron a cabo reuniones preliminares para esbozar las características deseadas y el alcance del proyecto, teniendo siempre en mente la experiencia de usuario y la eficiencia en la corrección y retroalimentación por parte del usuario.

3.1.2. Fase de análisis

Durante esta etapa se realizó una exploración profunda y meticulosa de las necesidades y los requerimientos clave para diseñar una plataforma de aprendizaje eficaz. Se buscó conseguir un entorno que proporcionará una buena retroalimentación, poniendo énfasis en un enfoque adaptativo y personalizado, teniendo en cuenta la diversidad de ritmos y estilos de aprendizaje. Sé investigaron tanto las demandas funcionales como las tecnológicas, y se realizó un análisis de la plataforma Codelearn S.L. con el fin de garantizar que las decisiones tomadas fueran adecuadas y eficientes.

Cabe destacar que antes de la construcción de la plataforma, fue esencial definir con precisión qué funcionalidades eran fundamentales. A través de

reuniones, lecturas e investigación se definieron las siguientes características principales para el sistema:

- Un sistema de corrección automatizada que evalúe tanto la solución como la calidad del código.
- Un sistema de retroalimentación que ofrezca comentarios constructivos.
- Un camino de ejercicios que se adapte dinámicamente al progreso y habilidades del estudiante.
- Un ranking para motivar a los estudiantes durante su aprendizaje.
- Una plataforma sencilla y visual, diseñada específicamente para niños, que les permita interactuar con el contenido de manera intuitiva.
- Un compilador que permita la ejecución y prueba de código en la propia web.
- Un sistema de soporte y tutoría basado en un mecanismo de preguntas y respuestas para asistir a los estudiantes con dudas específicas.

Cuando los requisitos y objetivos ya estuvieron claros, se pasó a la elección de tecnologías que se iban a usar. La decisión fue tomada en base la experiencia, ya que de esta manera se conseguía rápidamente un prototipo manejable.

Para el Frontend, se decidió el uso de HTML, CSS y JavaScript, debido la posibilidad de creación de interfaces de usuario fluidas y dinámicas. En cambio, para el Backend se usó Flask, por su fácil adaptabilidad con diversas librerías de Python. Para la gestión de la base de datos, se optó por PostgreSQL, ya que garantizaba una estructura robusta y escalable.

3.1.3. Fase de desarrollo

La fase de desarrollo se centró en implementar las características clave identificadas durante la fase anterior. Las principales características desarrolladas fueron:

1. **Corrección automatizada de los ejercicios**: Se desarrolló un sistema de corrección automatizada que no solo verifica la solución del ejercicio, sino también la calidad y eficiencia del código. Concretamente, analiza la limpieza del código, la utilización correcta de las estructuras de datos y si existen bucles repetitivos o redundantes. Garantizando que los estudiantes no solo resuelven el problema, sino que también adopten buenas prácticas de codificación.

2. **Retroalimentación Personaliza**: Se debe ofrecer un buen *feedback* para que los estudiantes entiendan sus errores. Por ello, se proporcionan comentarios y algunas sugerencias necesarias. De esta manera, se fortalece la comprensión y se facilita el aprendizaje.

3. **Caminos de Ejercicios Aleatorio**: Para poder proporcionar una buena experiencia de aprendizaje, el sistema tiene un conjunto de ejercicios basados en requisitos específicos de forma aleatoria. De esta manera, se consigue que cada vez que el estudiante se enfrenta un requisito, se le desafía con un conjunto de ejercicios diferentes, asegurando que los haga por comprensión y no por repetición. Es importante destacar, que si un estudiante no realiza correctamente un ejercicio, el sistema le proporcionará otro diferente para evitar frustración.

Cabe destacar que esta fase también implicó una serie de pruebas iterativas y correcciones, permitiendo ajustes y mejoras en tiempo real en función del feedback recibido y las observaciones realizadas.

3.1.4. Fase de prueba y corrección

Tras finalizar la anterior fase, tocó centrarse en el testeo del prototipo y sus respectivos ajustes. A pesar de haber diseñado la página web en base la investigación teórica, se debía validar su usabilidad y efectividad de las funciones realizadas.

Se eligió un grupo de 2-3 estudiantes, para que interactuaran con la plataforma. Pidiéndoles que hicieran varias tareas y que compartieran sus impresiones, dificultades y recomendaciones.

La retroalimentación de estos estudiantes fue clave. A partir de sus comentarios, se pudieron efectuar mejoras importantes en la plataforma. Se solucionaron fallos técnicos y cuestiones de usabilidad, además de optimizar la calidad de la experiencia educativa. De esta manera, se consiguió que la plataforma se adaptará a las necesidades y retos de los estudiantes mejor, pudiendo asegurar una experiencia educativa de calidad.

3.1.5. Elaboración de la memoria

Durante las fases previas del proyecto, se llevó a cabo una recolección de datos e información pertinente. Al aproximarnos a las etapas finales del desarrollo, dicha información fue estructurada y articulada de manera coherente y sistemática, siendo la creación de la memoria.

El proyecto tuvo inicio a principios de julio, durando cinco meses hasta su finalización. Es importante subrayar que el período previo a julio, aunque incluyó actividades de investigación y lectura, no es considerado crucial para la estructura del proyecto. Esto se debe a que, a pesar de haber realizado dichas tareas, no se le dedicó una cantidad de tiempo significante debido a

otras responsabilidades. Por esta razón, se presenta el diagrama de Gantt 3.1 que refleja el periodo de desarrollo del proyecto:

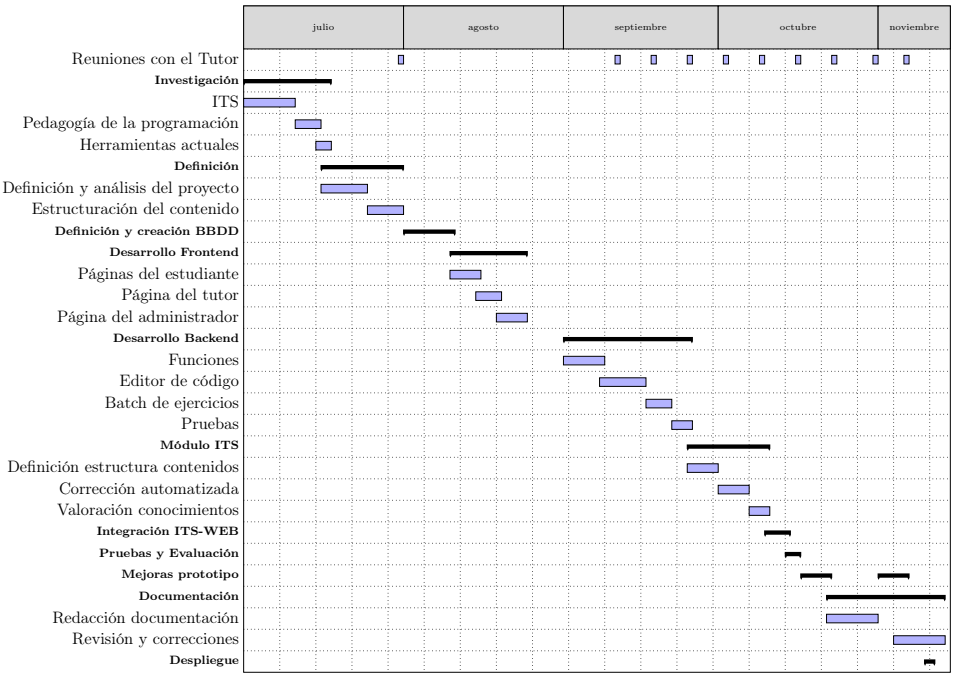

Figura 3.1: Diagrama de Gantt con la planificación del TFG

3.2. Presupuesto

En esta parte del documento se realiza una descripción de los gastos ocasionados por el proyecto, formando el denominado presupuesto 3.2.

3.2.1. Licencias

Con tal de realizar una buena evaluación del coste de un proyecto, se tuvieron en cuenta todos aquellos gastos relacionados con licencias. Por ello, se optó por maximizar el uso de tecnologías *OpenSource* para reducir costos.

Para la interfaz de usuario, se optó por usar HTML, CSS y JavaScript por su costo cero. En cuanto al Backend, se eligió Flask por el mismo motivo. Además, se recurrió a PostgreSQL que no tiene ningún coste ni límite en cuanto a tamaño de la base de datos. Para el control de versiones, se usó Git y GitHub [62]. Aunque GitHub ofrece planes de pago, dispone de buenas opciones gratuitas.

Por lo tanto, el coste total de las licencias es cero.

3.2.2. Recursos Materiales

Dentro de la inversión material, es indispensable considerar el equipo informático empleado en el proyecto, en este caso, un ordenador portátil tal como se puede ver en el apéndice A. Comúnmente, estos dispositivos tienen una vida útil estimada que oscila entre los 2 y 4 años.

Si asumimos una vida útil intermedia de 3 años para el dispositivo, se estimará una depreciación anual del equipo. Por ello, considerando que el portátil tuvo un precio inicial de 950€, su depreciación anual será de 316,67€.

Finalmente, se debe tener en cuenta que el proyecto se ha desarrollado en una duración aproximada de 5 meses, por ello, el coste asociado al desgaste del portátil durante dicho período es de 132,78€.

3.2.3. Costes de Personal

Para el cálculo de costes de personal se ha tenido en cuenta un único empleado: un ingeniero informático. Tomando los datos obtenidos de la Seguridad Social para el año 2023 [52], el salario de dicho ingeniero oscila entre una base mínima de cotización de 1.759,50€ y una base máxima de 4.495,50€ mensuales 3.1.

Concepto	**Coste Mensual (€)**	**Coste 5 Meses (€)**
Base Mínima de Cotización	1.759,50	8.797,50
Cotización Empresa (Base Mínima)	415,44	2.077,20
Cotización Trabajador (Base Mínima)	82,70	413,50
Total (Base Mínima)	**2.257,64**	**11.288,20**
Base Máxima de Cotización	4.495,50	22.477,50
Cotización Empresa (Base Máxima)	1.060,94	5.304,70
Cotización Trabajador (Base Máxima)	211,29	1.056,45
Total (Base Máxima)	**5.767,73**	**28.838,65**

Tabla 3.1: Costes asociados al personal para un ingeniero informático.

Si calculamos el promedio salarial, se obtendría aproximadamente unos 3.127,50€. Como el proyecto tiene una duración de 5 meses, el coste total en salarios sería de 15.637,50€ para el desarrollo completo del proyecto.

Adicionalmente, se debería considerar las cotizaciones correspondientes a la empresa y al trabajador, siendo el coste total para el proyecto de unos 17.900€ aproximadamente.

3.2.4. Otros costes

Dentro de este apartado se hace alusión a todos aquellos gastos indirectos importantes para la realización del proyecto. Por ejemplo, el material de

oficina, la conexión a Internet, la electricidad, los gastos de transporte, entre otros.

Este tipo de costes indirectos se suele estimar como un porcentaje del gasto principal, en este caso, del coste de personal. Siguiendo esta norma, se ha determinado que estos gastos representan aproximadamente el 10 % del gasto total en personal:

$$17.900 \times 0,10 = 1.790 \tag{3.1}$$

Por lo tanto, se estima un coste adicional de 1.790€ para cubrir lo gastos indirectos asociados al proyecto.

3.2.5. Total

Para finalizar, se muestra una tabla 3.2 que resume todos los costes obtenidos, haciendo una estimación mensual y total del proyecto.

Concepto	**Coste Mensual (€)**	**Coste Total (5 Meses) (€)**
Licencias	0,00	0,00
Recursos Materiales (Portátil)	26,56	132,78
Personal (Salario promedio)	3.127,50	15.637,50
Cotizaciones (aprox.)	452,50	2.262,50
Otros costes	358,00	1.790,00
Total	**3964,06**	**19.822,78**

Tabla 3.2: Total de costes del proyecto

Capítulo 4

Análisis

Dentro de este capítulo, se abordan diversos aspectos clave relacionados con la fase de análisis del proyecto. El enfoque principal es el análisis detallado de los usuarios objetivo 4.1, donde se exploran sus necesidades, expectativas y limitaciones. Este análisis es esencial para comprender en profundidad el contexto y los requerimientos del sistema.

Además, se presentan los modelos de caso de uso 4.3, que describen las interacciones de los usuarios con el sistema y ayudan a identificar las funcionalidades clave que debe incorporar la solución.

Se incluyen igualmente los modelos de comportamiento y el modelado de entidades y relaciones 4.4. Estos componentes son cruciales para entender la dinámica del sistema y la estructura de datos que soportará.

Al final de este capítulo, se habrá ofrecido una visión estructurada de la solución propuesta, sentando las bases para la obtención de los requisitos necesarios y facilitando el proceso de diseño.

4.1. Análisis de Usuarios Objetivo

El enfoque de la plataforma web está dirigido principalmente a niños entre los 8 y 18 años, siguiendo la metodología de CODELEARN S.L.. Donde la mayoría de los clientes son estudiantes con poco o ningún conocimiento previo de programación. Es por ello que la plataforma está diseñada para ser intuitiva y flexible, en línea con las recomendaciones de Large y Beheshti sobre el diseño de interfaces para niños [37].

Si tenemos en cuenta la brevedad del período de atención de los niños y su desarrollo cognitivo en curso, se deberán evitar tareas extensas o excesivamente complicadas. Las expectativas y necesidades son diversas, pero comúnmente incluyen una interfaz visualmente atractiva, instrucciones claras y una retroalimentación inmediata tal como especifican Duglas Frye y Elliot Soloway [22]. Asimismo, se deben incorporar elementos de gamificación dentro la plataforma con tal mantener el interés y la motivación de los usuarios.

Finalmente, la plataforma debe ser accesible; es decir, se podrá acceder desde múltiples dispositivos como ordenadores, tablets y smartphones debido a su diseño *Web Responsive*.

4.2. Análisis de Requisitos Educativos

La enseñanza de habilidades de programación dentro de las escuelas está cobrando una gran relevancia [55]. Sin embargo, hay una falta de herramientas y enfoques efectivos mucho más allá de herramientas como Scratch [55]. Conscientes de esta brecha, se adopta un enfoque fundamentado en evidencia y prácticas académicas sólidas.

Aunque lenguajes como Scratch y Logo son de los más populares para la educación de niños, se opta por Python, C++, Java y HTML+CSS+JS. Python es conocido por su simplicidad y legibilidad, lo que facilita el aprendizaje para los principiantes [45]. Además, permite una transición fluida a lenguajes más complejos. C++, Java y HTML+CSS+JS, aunque sujetos a debate sobre su idoneidad para principiantes, son cruciales en la industria y la educación. Siendo su relevancia práctica la justificación de su presencia [13].

Al ir más allá de los enfoques basados en bloques como Scratch, se busca que los niños aprendan la programación basada en texto, más cercana a la realidad. Los ejercicios se organizan por competencias específicas, permitiendo un aprendizaje adaptable y progresivo. Alineando el enfoque con los resultados de investigaciones sobre el tema [50].

Es importante destacar que el sistema debe ser inclusivo, adaptando los ejercicios al progreso individual del estudiante y con un buen sistema de asistencia. De esta manera, se siguen las directrices recomendadas por Luxton-Reily y Wünsche sobre las características de un buen sistema inteligente para la enseñanza de programación [11], consiguiendo un buen enfoque educativo.

4.3. Modelos de Caso de Uso y Escenarios de Uso

Dentro de esta sección se va a hablar de las interacciones usuario-sistema. En primer lugar, se abordará los modelos de caso de uso 4.1. Debido a que nos proporcionan una visión panorámica, resaltando las posibles interacciones y los actores implicados. Es decir, ayudará a definir lo que el sistema puede hacer y lo que se espera que haga. Además, también se va a especificar los diferentes escenarios de uso, ya que proporcionarán una mejor comprensión de las interacciones específicas pudiendo observar las verdaderas dinámicas del sistema en acción.

Con esta combinación, se establecen las funcionalidades esperadas del sistema y cómo este responderá ante diferentes situaciones y acciones del

usuario. Es importante destacar que todos los diagramas que se ven durante la memoria se han realizado mediante PlantULM [1].

4.3.1. Actores del Sistema

La interacción con la plataforma se basa en tres tipos de usuarios; estudiantes, profesores y administradores. El acceso a la plataforma está restringido a los usuarios registrados, ya que se busca asegurar que cada interacción dentro del sistema se realice en un entorno seguro y personalizado. De esta manera, el sistema es capaz de ofrecer una experiencia adaptada al estudiante y a su progreso.

Los actores identificados en el sistema son:

- **Estudiantes**: Constituyen el grupo principal de usuarios. Su interacción con la plataforma está enfocada en el aprendizaje de programación mediante los recursos educativos disponibles. Los estudiantes tienen acceso a ejercicios, material didáctico, retroalimentación personalizada y un apartado personal para la resolución de dudas.

- **Profesores**: Actúan como facilitadores del proceso de aprendizaje y brindan apoyo emocional a los estudiantes. Entre sus responsabilidades específicas está la corrección de ejercicios de HTML, que requieren una atención más detallada, así como la resolución de dudas y el seguimiento personalizado de estudiantes, especialmente a aquellos que presentan dificultades o inactividad, para motivarles y guiarles en su proceso educativo.

- **Administradores**: Son los usuarios con el mayor nivel de control dentro del sistema. Se encargan de la gestión integral del contenido didáctico y de la administración de los profesores. Su rol es esencial para el mantenimiento y la actualización constante de la plataforma.

El diagrama de caso de uso 4.1 que se muestra a continuación ilustra cómo estos actores interactúan con el sistema. Cada actor está asociado con unas acciones específicas y flujos de trabajo dentro de la plataforma, reflejando el enfoque personalizado y la adaptabilidad del sistema.

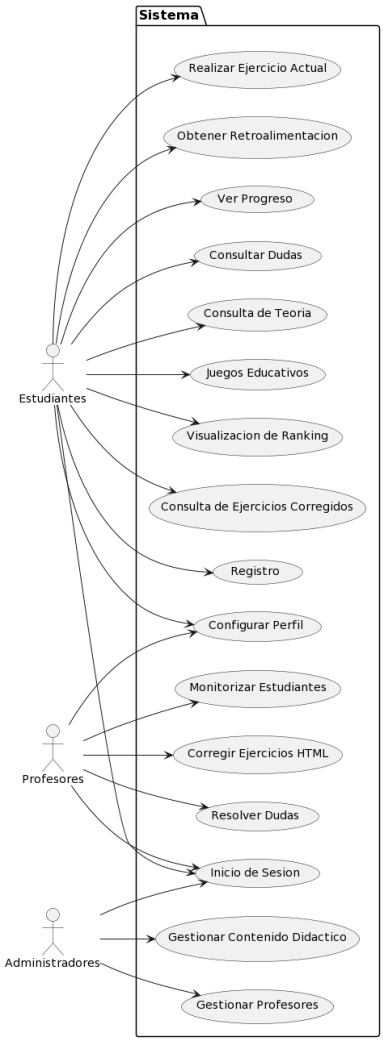

Figura 4.1: Diagrama de Caso de Uso del sistema

4.3.2. Escenarios de Uso Detallados

En esta subsección, se realiza un análisis de los escenarios de uso que involucran a los diferentes tipos de usuarios de la plataforma. Para cada escenario, se presentarán tablas detalladas que ofrecen una comprensión clara de cómo los usuarios interactúan con el sistema. Estos escenarios son esenciales para entender las funcionalidades y la respuesta del sistema ante las necesidades de los usuarios.

Los elementos que compondrán cada tabla de escenarios de uso son los siguientes, siguiendo las pautas recomendadas por expertos en usabilidad:

- **ID**: Código de identificación único para cada escenario de uso.

- **Nombre**: Título descriptivo para el escenario de uso.

- **Actor**: Definición de los usuarios implicados en la acción.

- **Descripción**: Breve explicación del escenario de uso.

- **Precondiciones**: Condiciones previas necesarias para que se inicie el escenario

- **Postcondición**: Estado o resultado alcanzado tras completar el escenario.

- **Flujo**:Detalle de todos los pasos del escenario, incluyendo flujos básicos y alternativos.

Los escenarios se centran en aspectos cruciales de la plataforma, enfatizando aquellos factores más relevantes y únicos. Se ha decidido omitir escenarios de uso comunes y autoexplicativos, como el inicio de sesión, registro o cierre de sesión, para centrarse en aquellos que aportan una mayor comprensión de las funcionalidades específicas y la interacción del usuario con la plataforma.

ID	EU-1
Nombre	Realización de un ejercicio
Actor	Estudiante
Descripción	El estudiante resuelve un ejercicio de programación.
Precondiciones	EEstudiante ha iniciado sesión y ha seleccionado un módulo pendiente.
Postcondición	El ejercicio se marca como completado y el estudiante recibe retroalimentación.
Flujo	1. El estudiante accede al módulo pendiente. 2. Se abre un ejercicio para realizar. 3. Resuelve el ejercicio dentro del tiempo establecido. 4. Envía la solución para su evaluación. 5. Recibe retroalimentación y puntuación.

Tabla 4.1: EU-1: Realización de un ejercicio

ID	EU-2
Nombre	Tiempo superado en un ejercicio
Actor	Estudiante
Descripción	El estudiante no logra completar un ejercicio de programación dentro del tiempo límite.
Precondiciones	Estudiante ha iniciado sesión y está resolviendo un ejercicio.
Postcondición	El ejercicio se marca como erróneo y se le muestra un nuevo ejercicio.
Flujo	1. Estudiante trabaja en un ejercicio. 2. No completa el ejercicio en el tiempo límite. 3. El sistema marca el ejercicio como fallido. 4. Se presenta automáticamente un nuevo ejercicio al estudiante.

Tabla 4.2: EU-2: Tiempo superado en un ejercicio

ID	EU-3
Nombre	Corrección de ejercicios por parte del profesor
Actor	Profesor
Descripción	El profesor corrige manualmente ejercicios HTML debido a su complejidad.
Precondiciones	Profesor ha iniciado sesión y tiene ejercicios HTML pendientes de corrección.
Postcondición	Ejercicio corregido y retroalimentación enviada al estudiante.
Flujo	1. Profesor revisa la lista de ejercicios HTML pendientes. 2. Selecciona un ejercicio para corregir. 3. Evalúa el ejercicio y redacta comentarios de retroalimentación. 4. Marca el ejercicio como corregido y envía la retroalimentación.

Tabla 4.3: EU-3: Corrección de ejercicios por parte del profesor

ID	EU-4
Nombre	Administrador gestionando contenido didáctico
Actor	Administrador
Descripción	El administrador añade, consulta edita o elimina contenido educativo en la plataforma.
Precondiciones	Administrador ha iniciado sesión.
Postcondición	Contenido educativo actualizado en la plataforma.
Flujo	1. Administrador accede al módulo de gestión de contenido. 2. Elige entre añadir, editar o eliminar contenido. 3. Realiza la acción seleccionada y revisa los cambios. 4. Confirma y actualiza el contenido en la plataforma.

Tabla 4.4: EU-4: Gestión del contenido didáctico

ID	EU-5
Nombre	Resolución de dudas
Actor	Estudiante, Profesor
Descripción	El estudiante plantea una duda y el profesor la resuelve.
Precondiciones	Estudiante y profesor han iniciado sesión.
Postcondición	Duda resuelta.
Flujo	1. Estudiante envía una pregunta en la sección de dudas. 2. Profesor revisa las dudas pendientes en su panel. 3. Selecciona y responde la duda del estudiante. 4. Estudiante recibe la respuesta y notificación de duda resuelta.

Tabla 4.5: EU-5: Resolución de dudas

ID	EU-6
Nombre	Seguimiento del Progreso del Estudiante
Actor	Profesor
Descripción	El profesor revisa y hace seguimiento del progreso de los estudiantes en sus ejercicios y módulos.
Precondiciones	Profesor ha iniciado sesión.
Postcondición	Comprensión actualizada del progreso del estudiante.
Flujo	1. Profesor accede al panel de seguimiento. 2. Selecciona un estudiante para revisar su progreso. 3. Analiza el rendimiento del estudiante en ejercicios y módulos.

Tabla 4.6: EU-6: Seguimiento del Progreso del Estudiante

ID	EU-7
Nombre	Gestión de Usuarios
Actor	Administrador
Descripción	El administrador gestiona las cuentas de usuario, incluyendo la creación, consulta, modificación y eliminación de cuentas de estudiantes y profesores.
Precondiciones	Administrador ha iniciado sesión.
Postcondición	Las cuentas de usuario son actualizadas según las acciones realizadas.
Flujo	1. Administrador accede al panel de gestión de usuarios. 2. Elige entre crear, modificar o eliminar cuentas. 3. Realiza la acción seleccionada y revisa los cambios. 4. Guarda los cambios y actualiza el registro de usuarios.

Tabla 4.7: EU-7: Gestión de usuarios

4.3.3. Restricciones y Reglas de Negocio

Esta subsección detalla las pautas operativas y limitaciones que rigen la interacción de los usuarios con el sistema. Se establecen restricciones específicas para cada tipo de usuario y se definen reglas clave que garantizan un entorno seguro y eficiente, delineando así las condiciones bajo las cuales opera la plataforma educativa.

Restricciones de Acceso

- **Estudiantes**: Su acceso se limita a los módulos y ejercicios específicamente asignados por la plataforma, garantizando un aprendizaje estructurado.
- **Profesores**: Capacidad de acceder a la corrección de ejercicios HTML, al panel de dudas y a consultar el progreso de los estudiantes, pero sin permisos para modificar el contenido didáctico.
- **Administradores**: Poseen control total sobre la gestión del contenido y las cuentas de los profesores, asegurando la calidad y la actualización constante de la plataforma.

Reglas de Validación

- Los ejercicios deben completarse dentro de un tiempo límite establecido; si se excede, se considerarán fallidos.
- La puntuación se asigna en base a la corrección y calidad del código entregado.

- Los ejercicios HTML requieren una validación manual específica por parte de los profesores.

Reglas de Negocio Adicionales

- Si un estudiante falla en un ejercicio, se le asigna un nuevo problema como medida para prevenir la desmotivación o frustración.
- Los profesores tienen la capacidad de identificar a estudiantes con bajo rendimiento o inactividad prolongada, y deben enviar correos motivacionales para apoyarlos.
- La autorización para añadir o eliminar cuentas de profesores está reservada exclusivamente a los administradores.
- Proceso de registro controlado para añadir estudiantes, sin posibilidad de inclusión directa por parte de administradores o profesores.
- Prohibición de borrado de datos de estudiantes, garantizando la integridad y privacidad de la información.

4.3.4. Seguridad y Acceso

Para garantizar la integridad de la plataforma, se deben abordar aspectos de seguridad y acceso.

Autenticación La plataforma adopta un sistema de autenticación, concretamente se logra mediante el uso de correo electrónico y contraseña.

Autorización Diferentes roles de usuario, como estudiantes, profesores y administradores, tienen niveles de acceso específicos. Por ejemplo, mientras que los profesores pueden corregir ejercicios y resolver dudas, no tienen autorización para modificar el contenido educativo.

Seguridad de Datos La protección de los datos es una alta prioridad. Las contraseñas se almacenan de forma segura, empleando técnicas modernas de hash y salting.

Monitorización y Auditoría Se lleva a cabo una monitorización constante de todas las actividades significativas dentro de la plataforma. Esto no solo permite a los administradores realizar seguimientos detallados, sino que también facilita la identificación y respuesta rápida ante cualquier actividad inusual o sospechosa, mejorando así la seguridad y la eficiencia del sistema.

4.4. Modelado de Datos y Diagrama Entidad-Relación

El modelado de datos es fundamental para entender cómo se van a estructurar y relacionar las diferentes entidades dentro de la base de datos.

En este proyecto, entidades como Estudiantes, Módulos, Requisitos, Ejercicios, Teorías, Preguntas, Juegos interactúan entre sí, formando el núcleo de la lógica operativa y las funcionalidades del sistema. Además, se introducen entidades adicionales que refuerzan y detallan aún más este modelo. El Diagrama Entidad-Relación (ER), presentado en la Figura 4.2, proporciona una representación visual detallada de estas interacciones.

En el diagrama, se destacan las siguientes entidades clave y sus roles específicos en el sistema:

- **Users**: Incluyen a estudiantes, profesores y administradores, cada uno interactuando con el sistema de manera única.
- **Questions**: Representan consultas o dudas planteadas por los estudiantes, vinculadas a ejercicios o teorías específicas.
- **Exercises**: Tareas individuales que están integradas en un requisito de un módulo concreto, representando desafíos prácticos para los estudiantes.
- **Theory**: Material didáctico vinculado a cada requisito de un módulo, proporcionando el marco teórico necesario.
- **Modules**: Agrupan ejercicios y teoría, estructurando el contenido educativo en torno a temas específicos de programación.
- **Requirements**: Establecen los objetivos educativos para los módulos, siendo cada requisito un elemento concreto de la programación que se quiere enseñar.
- **Games**: Actividades de gamificación diseñadas para mejorar la experiencia de aprendizaje y motivar a los estudiantes.
- **Payments**: Gestionan las transacciones en las que los estudiantes usan puntos acumulados para adquirir juegos.
- **Notifications**: Gestiona las alertas y mensajes que los estudiantes reciben, vital para la comunicación y el seguimiento del estudiante.
- **StudentProgress y StudentActivity**: Registra el avance y la actividad de cada estudiante en el sistema, lo que permite un seguimiento personalizable y adaptable de su aprendizaje.
- **TheoryRequirements y ExerciseRequirements**: Define los requisitos previos para acceder a ciertos ejercicios y materiales teóricos, asegurando una progresión lógica en el aprendizaje.

- **ExtraExercises**: Si es pertinente, se almacena por estudiante unos ejercicios concretos que debe realizar.
- **StudentModules**: Relacionan a los estudiantes con los módulos específicos donde se encuentran.
- **ModuleRequirementsOrder**: Establecen un orden lógico para los módulos y sus requisitos, garantizando una estructura educativa coherente.
- **UserRequirementsCompleted**: Registran los requisitos completados por los usuarios.

Las relaciones entre estas entidades son esenciales para el funcionamiento eficaz del sistema. Por ejemplo, los módulos están formados por requisitos que contienen ejercicios y teoría, formando un plan de estudios coherente. Los juegos añaden un elemento interactivo y lúdico, mientras que los registros de pagos controlan el uso de puntos para el acceso a estos. Las preguntas, por otro lado, facilitan la interacción directa entre estudiantes y profesores, enriqueciendo el proceso de aprendizaje.

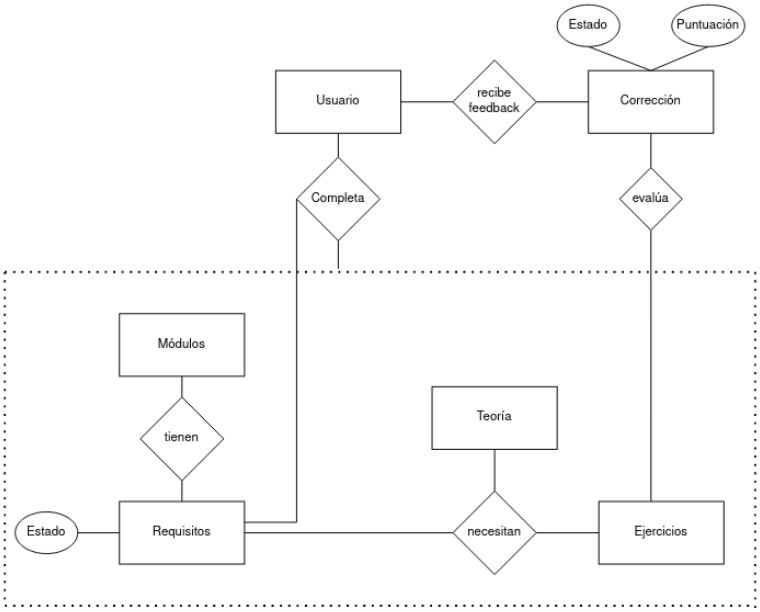

Figura 4.2: Diagrama Entidad-Relación del sistema

Capítulo 5

Requisitos

Basado en el análisis presentado anteriormente, este capítulo se dedica a perfilar los requisitos de la plataforma. La evaluación previa ha establecido un fundamento detallar dichos requisitos. Estos serán el eje que dirigirá el proceso de desarrollo, con el propósito de garantizar que la plataforma resultante satisfaga plenamente las expectativas y requisitos de los usuarios finales.

5.1. Requisitos Funcionales

Con tal de garantizar la funcionalidad del programa y la realización de los objetivos establecidos, es esencial tener una buena definición de los requisitos funcionales. Para facilitar su comprensión, presentaremos estos requisitos en tablas, siguiendo las convenciones de la ingeniería de requisitos y desarrollo de software. La organización de los contenidos es la siguiente:

- **ID**: Identificador único para cada requisito. Esencial para una fácil referencia y seguimiento.
- **Nombre**: Nombre del requisito.
- **Descripción**: Explicación del propósito y alcance del requisito.
- **Prioridad**: La importancia del requisito en relación con el sistema. Las categorías posibles son alta, media o baja. Siendo crucial abordar primero los requisitos de alta prioridad, mientras que los de baja prioridad se harán al final.
- **Estado**: Refleja la situación actual del requisito, pudiendo estar *en desarrollo*, *completado*, *en revisión* o *rechazado*.

5.1.1. Generales

ID	RF-01
Nombre	Inicio de sesión
Descripción	Los usuarios registrados deben poder acceder a la plataforma introduciendo su nombre de usuario y contraseña. El sistema debe garantizar la confidencialidad y autenticidad de esta operación, asegurando que las contraseñas se gestionan y almacenan de manera segura.
Prioridad	Alta
Estado	Completado

Tabla 5.1: Requisito funcional RF-01: Inicio de sesión.

ID	RF-02
Nombre	Registro de usuario
Descripción	Los nuevos usuarios deben poder registrarse en la plataforma proporcionando información relevante.
Prioridad	Alta
Estado	Completado

Tabla 5.2: Requisito funcional RF-02: Registro de usuario.

ID	RF-03
Nombre	Perfil de usuario
Descripción	Cada usuario debe tener un perfil donde pueda visualizar su información personal, y donde poder cambiar ciertos datos.
Prioridad	Media
Estado	Completado

Tabla 5.3: Requisito funcional RF-03: Perfil de usuario.

5.1.2. Sistema

ID	RF-04
Nombre	Selección de ejercicios por requisitos
Descripción	El sistema deberá presentar a los estudiantes ejercicios basados en el requisito que estén estudiando. Una vez completado un ejercicio, el siguiente deberá ser mostrado de forma aleatoria dentro del mismo requisito, hasta que se considere que el estudiante ha asimilado ese requisito.
Prioridad	Alta
Estado	Completado

Tabla 5.4: Requisito funcional RF-04: Selección de ejercicios por requisitos.

ID	RF-05
Nombre	Corrección de ejercicios
Descripción	Al enviar una solución, el sistema no solo verificará si es correcta, sino que también evaluará la calidad del código en términos de optimización y buenas prácticas de programación.
Prioridad	Alta
Estado	Completado

Tabla 5.5: Requisito funcional RF-05: Corrección de ejercicios.

ID	RF-06
Nombre	Compilador integrado
Descripción	La plataforma deberá contener con un compilador integrado que permita a los estudiantes probar sus códigos en Java, HTML+CSS+JS, Python y C++.
Prioridad	Alta
Estado	Completado

Tabla 5.6: Requisito funcional RF-06: Compilador integrado.

ID	RF-07
Nombre	Categorización de la teoría
Descripción	La teoría se clasificará en función del requisito, facilitando así su visualización dependiendo del punto en que se encuentre el estudiante.
Prioridad	Media
Estado	Completado

Tabla 5.7: Requisito funcional RF-07: Categorización de la teoría.

ID	RF-08
Nombre	Repetición de ejercicios similares tras fallo
Descripción	Si un estudiante falla un ejercicio, el sistema deberá presentar otro ejercicio con características similares para que el estudiante pueda intentarlo de nuevo.
Prioridad	Alta
Estado	Completado

Tabla 5.8: Requisito funcional RF-08: Repetición de ejercicios similares tras fallo.

ID	RF-09
Nombre	Límite de tiempo para completar un ejercicio
Descripción	Si el estudiante tarda más de un tiempo predefinido en completar un ejercicio, el sistema lo marcará como erróneo y presentará un nuevo ejercicio.
Prioridad	Media
Estado	Completado

Tabla 5.9: Requisito funcional RF-09: Límite de tiempo para completar un ejercicio.

5.1.3. Estudiantes

ID	RF-10
Nombre	Resolución de dudas
Descripción	Los estudiantes realizarán preguntas y adjuntar archivos para que el profesor les ayude.
Prioridad	Alta
Estado	Completado

Tabla 5.10: Requisito funcional RF-10: Resolución de dudas.

ID	RF-11
Nombre	Consulta de los rankings
Descripción	Los estudiantes podrán consultar un ranking donde vean los *top* 5 estudiantes diarios, semanales y mensuales.
Prioridad	Media
Estado	Completado

Tabla 5.11: Requisito funcional RF-11: Consulta de los rankings.

ID	RF-12
Nombre	Juegos en la web
Descripción	Por cada módulo que los estudiantes completen, se les desbloqueará un juego como recompensa.
Prioridad	Baja
Estado	Completado

Tabla 5.12: Requisito funcional RF-12: Juegos en la web.

ID	RF-13
Nombre	Consulta de los ejercicios corregidos
Descripción	Los estudiantes podrán consultar los comentarios del profesor sobre los ejercicios que haya corregido.
Prioridad	Media
Estado	Completado

Tabla 5.13: Requisito funcional RF-13: Consulta de los ejercicios corregidos.

ID	RF-14
Nombre	Sistema de notificación
Descripción	Se notificará al estudiante si su ejercicio es correcto o incorrecto cuando lo evalúe el profesor, junto con una retroalimentación del por qué es incorrecto.
Prioridad	Alta
Estado	Completado

Tabla 5.14: Requisito funcional RF-14: Sistema de notificación.

ID	RF-15
Nombre	Feedback sobre la solución
Descripción	Cuando un alumno envíe la solución, se le mostrará un feedback completo sobre ella.
Prioridad	Alta
Estado	Completado

Tabla 5.15: Requisito funcional RF-15: Feedback sobre la solución.

ID	RF-16
Nombre	Sistema de puntos
Descripción	Por cada ejercicio completado, el estudiante obtendrá una cantidad de puntos dependiendo de la calidad de su solución.
Prioridad	Baja
Estado	Completado

Tabla 5.16: Requisito funcional RF-16: Sistema de puntos.

5.1.4. Profesores

ID	RF-17
Nombre	Lista de usuarios
Descripción	El profesor podrá ver una lista de todos los usuarios registrados en la plataforma.
Prioridad	Media
Estado	Completado

Tabla 5.17: Requisito funcional RF-17: Lista de usuarios.

ID	RF-18
Nombre	Consulta de ejercicios realizados
Descripción	El profesor podrá consultar los ejercicios realizados por los estudiantes, junto con su estado,
Prioridad	Media
Estado	Completado

Tabla 5.18: Requisito funcional RF-18: Consulta de ejercicios realizados.

ID	RF-19
Nombre	Panel de resolución de preguntas
Descripción	El profesor tendrá un panel donde podrá responder las preguntas realizadas por los estudiantes.
Prioridad	Alta
Estado	Completado

Tabla 5.19: Requisito funcional RF-19: Panel de resolución de preguntas.

ID	RF-20
Nombre	Lista de estudiantes inactivos
Descripción	El profesor podrá consultar en una lista los estudiantes que llevan más de una semana sin conectarse, incluyendo sus correos electrónicos para un posible seguimiento.
Prioridad	Media
Estado	Completado

Tabla 5.20: Requisito funcional RF-20: Lista de estudiantes inactivos.

ID	RF-21
Nombre	Tiempo de resolución de ejercicios
Descripción	Habrá una lista de estudiantes que tengan un tiempo de resolución de ejercicios demasiado bajo o alto, con tal de identificar posibles problemas o áreas de mejora.
Prioridad	Baja
Estado	Completado

Tabla 5.21: Requisito funcional RF-21: Tiempo de resolución de ejercicios.

ID	RF-22
Nombre	Corrección de ejercicios
Descripción	El profesor tendrá la capacidad de corregir los ejercicios, relacionados con HTML, enviados por los estudiantes y ser capaz de proporcionar una retroalimentación.
Prioridad	Alta
Estado	Completado

Tabla 5.22: Requisito funcional RF-22: Corrección de ejercicios.

ID	RF-23
Nombre	Usuarios con múltiples fallos
Descripción	Se listarán los usuarios que han fallado más de dos veces un ejercicio específico con tal de poder identificar áreas problemáticas.
Prioridad	Baja
Estado	Completado

Tabla 5.23: Requisito funcional RF-23: Usuarios con múltiples fallos.

ID	RF-24
Nombre	Ejercicios realizados por día
Descripción	Habrá un gráfico donde se podrá consultar el número de ejercicios realizados por día en la plataforma. Con tal de poder evaluar la actividad y el compromiso de los estudiantes.
Prioridad	Baja
Estado	Completado

Tabla 5.24: Requisito funcional RF-24: Ejercicios realizados por día.

ID	RF-25
Nombre	Usuarios con alta tasa de errores
Descripción	Habrá una lista de usuarios con una alta tasa de errores en los ejercicios, con tal de poder ofrecer asistencia adicional.
Prioridad	Baja
Estado	Completado

Tabla 5.25: Requisito funcional RF-25: Usuarios con alta tasa de errores.

5.1.5. Administrador

ID	RF-26
Nombre	Alta de profesor por administrador
Descripción	El administrador tendrá la capacidad de registrar a un profesor mediante el panel de administración.
Prioridad	Alta
Estado	Completado

Tabla 5.26: Requisito funcional RF-26: Alta de profesor por administrador.

ID	RF-27
Nombre	Añadir elementos
Descripción	El administrador podrá añadir nuevos módulos, requisitos, profesores, teoría y ejercicios a la plataforma.
Prioridad	Alta
Estado	Completado

Tabla 5.27: Requisito funcional RF-27: Añadir elementos.

ID	RF-28
Nombre	Consultar elementos
Descripción	El administrador podrá consultar los detalles de los módulos, requisitos, profesores, teoría y ejercicios existentes en la plataforma.
Prioridad	Alta
Estado	Completado

Tabla 5.28: Requisito funcional RF-28: Consultar elementos.

ID	RF-29
Nombre	Editar elementos
Descripción	El administrador podrá editar los detalles de los módulos, profesores, teoría y ejercicios, excepto los requisitos que no son editables.
Prioridad	Alta
Estado	Completado

Tabla 5.29: Requisito funcional RF-29: Editar elementos.

ID	RF-30
Nombre	Eliminar elementos
Descripción	El administrador podrá eliminar módulos, requisitos, profesores, teoría y ejercicios de la plataforma.
Prioridad	Alta
Estado	Completado

Tabla 5.30: Requisito funcional RF-30: Eliminar elementos.

ID	RF-31
Nombre	Orden global
Descripción	El administrador podrá crear y consultar un orden global que especifica el camino que debe seguir el estudiante en la plataforma, siendo una secuencia de módulos y requisitos.
Prioridad	Alta
Estado	Completado

Tabla 5.31: Requisito funcional RF-31: Orden global.

5.2. Requisitos No Funcionales

Con tal de asegurar la calidad global del sistema, deberemos tener en cuenta los requisitos no funcionales. Por ello, se van a detallar estos requisitos de manera estructurada siguiendo el mismo formato que con los requisitos funcionales.

ID	RNF-01
Nombre	Usabilidad
Descripción	El sistema deberá ser intuitivo y fácil de usar para los diferentes tipos de usuarios.
Prioridad	Alta
Estado	Completado

Tabla 5.32: Requisito no funcional RNF-01: Usabilidad.

ID	RNF-02
Nombre	Rendimiento
Descripción	El sistema deberá ser capaz de manejar una cantidad promedio de usuarios.
Prioridad	Media
Estado	Completado

Tabla 5.33: Requisito no funcional RNF-02: Rendimiento.

ID	RNF-03
Nombre	Seguridad
Descripción	Todos los datos sensibles, como contraseñas, deben estar encriptados.
Prioridad	Alta
Estado	Completado

Tabla 5.34: Requisito no funcional RNF-03: Seguridad.

ID	RNF-04
Nombre	Compatibilidad
Descripción	El sistema debe ser compatible con los navegadores web más comunes, incluidos Chrome, Firefox y Safari.
Prioridad	Alta
Estado	Completado

Tabla 5.35: Requisito no funcional RNF-04: Compatibilidad.

ID	RNF-05
Nombre	Escalabilidad
Descripción	El sistema deberá ser escalable para permitir la adición de más módulos, ejercicios y usuarios en el futuro.
Prioridad	Alta
Estado	Completado

Tabla 5.36: Requisito no funcional RNF-05: Escalabilidad.

ID	RNF-06
Nombre	Disponibilidad
Descripción	La página web debe estar disponible el 99 % del tiempo, excluyendo el tiempo de mantenimiento programado.
Prioridad	Media
Estado	Completado

Tabla 5.37: Requisito no funcional RNF-06: Disponibilidad.

ID	RNF-07
Nombre	Tiempo de Respuesta
Descripción	El tiempo de carga de las páginas no debe superar los 5 segundos.
Prioridad	Media
Estado	Completado

Tabla 5.38: Requisito no funcional RNF-07: Tiempo de Respuesta.

ID	RNF-08
Nombre	Mantenimiento
Descripción	Se deberá tener una arquitectura bien estructurada que facilite poder añadir nuevas funcionalidades. Además, de poder corregir errores sin afectar al funcionamiento de todo el sistema
Prioridad	Alta
Estado	Completado

Tabla 5.39: Requisito no funcional RNF-08: Mantenimiento.

5.3. Requisitos de Información

Dentro del marco de requisitos esenciales para el desarrollo de la plataforma, los requisitos de información también tienen su papel. Estos requisitos deben definir cómo debe ser recolectada, almacenada, procesada y protegida la información dentro del sistema.

La organización de los contenidos de los requisitos de información es la siguiente:

- **ID**: Identificador único para cada requisito. Esencial para una fácil referencia y seguimiento durante todo el ciclo de vida del desarrollo del software.

- **Nombre**: El nombre del requisito, que se corresponde directamente con cada entidad de la base de datos para facilitar la correlación.

- **Descripción**: Una explicación del tipo de datos dentro del sistema.

- **Contenido**: Detalles específicos sobre la información que se almacena, incluyendo formatos, estructuras y cualquier restricción relevante.

ID	RI-01
Nombre	Usuario
Descripción	Datos esenciales de los usuarios para permitir el acceso y la interacción con el sistema.
Contenido	Información básica del usuario como nombre, apellidos, fecha de nacimiento, contacto y credenciales de acceso. También se incluye información para el seguimiento del progreso educativo y la interacción con el sistema.

Tabla 5.40: Requisito de información RI-01: Usuario.

ID	RI-02
Nombre	Teoría
Descripción	El sistema almacenará información educativa asociada con los distintos módulos de aprendizaje.
Contenido	Incluye el contenido textual de la teoría, identificadores de los módulos relacionados y rutas a recursos gráficos. Se debe mantener la integridad referencial con los módulos del curso.

Tabla 5.41: Requisito de información RI-02: Teoría.

ID	RI-03
Nombre	Ejercicios
Descripción	Conjunto de datos que definen los ejercicios prácticos dentro de los módulos del sistema.
Contenido	Información detallada de los ejercicios, como el nombre, contenido descriptivo, soluciones y parámetros de evaluación, así como la identificación de si requieren revisión manual o son ejercicios clave dentro de los módulos.

Tabla 5.42: Requisito de información RI-03: Ejercicios.

ID	RI-04
Nombre	Juegos
Descripción	Datos relacionados con los juegos disponibles en la plataforma.
Contenido	Título de cada juego y su costo asociado, que puede utilizarse para transacciones o recompensas dentro del sistema educativo de la plataforma.

Tabla 5.43: Requisito de información RI-04: Juegos.

ID	RI-05
Nombre	Progreso del estudiante
Descripción	Registro detallado del avance de cada estudiante en los ejercicios y módulos del curso.
Contenido	Información de progreso que incluye el estado del ejercicio (completado, en progreso, etc.), calificaciones, fechas relevantes, código de solución proporcionado por el estudiante, y cualquier comentario del instructor o puntuación de evaluación automática.

Tabla 5.44: Requisito de información RI-05: Progreso del estudiante.

ID	RI-06
Nombre	Actividad del estudiante
Descripción	Información que captura las interacciones específicas de los estudiantes con los contenidos del curso.
Contenido	Datos sobre las acciones realizadas por el estudiante dentro de la plataforma, incluyendo si han completado o saltado actividades y el tipo de contenido con el que interactuaron, para proporcionar una visión integral de su compromiso con el material del curso.

Tabla 5.45: Requisito de información RI-06: Actividad del estudiante.

ID	RI-07
Nombre	Transacciones de Pagos
Descripción	Registros de pagos realizados por los usuarios, que detallan la adquisición de los juegos.
Contenido	Detalles de cada transacción, incluyendo el usuario que realiza el pago, el juego o servicio adquirido, la cantidad pagada y la fecha del pago.

Tabla 5.46: Requisito de información RI-07: Transacciones de Pagos.

ID	RI-08
Nombre	Notificaciones
Descripción	Captura la información de las notificaciones generadas por el sistema o los profesores para los usuarios.
Contenido	Datos de las notificaciones como el destinatario, el mensaje específico enviado, la fecha y hora de envío, y un indicador de si la notificación ha sido leída.

Tabla 5.47: Requisito de información RI-08: Notificaciones.

ID	RI-09
Nombre	Requisitos
Descripción	Información relativa a los requisitos educativos que deben ser cumplidos por los estudiantes en el transcurso de su aprendizaje. Estos requisitos son elementos fundamentales que estructuran los módulos y el contenido del curso.
Contenido	Datos que incluyen un identificador único y el nombre del requisito, que sirve como referencia para la progresión en los módulos y la vinculación con ejercicios y teoría.

Tabla 5.48: Requisito de información RI-09: Requisitos.

ID	RI-10
Nombre	*Path* educativo
Descripción	Detalles que establecen la secuencia en la que los estudiantes deben cumplir con los requisitos dentro de un módulo específico. Esta secuencia asegura que el proceso de aprendizaje siga una progresión lógica y estructurada.
Contenido	Datos que incluyen el identificador del módulo, el identificador del requisito y la posición que ocupa este requisito en la secuencia de aprendizaje del módulo.

Tabla 5.49: Requisito de información RI-10: *Path* educativo.

ID	RI-11
Nombre	Módulo
Descripción	Información relacionada con los módulos que componen la estructura curricular de la plataforma. Cada módulo agrupa una serie de requisitos, que a su vez agrupan ejercicios y teoría para facilitar el aprendizaje en temas específicos de la programación.
Contenido	Datos que definen cada módulo, incluyendo un identificador único, el nombre del módulo y una descripción que detalla los objetivos y el contenido cubierto.

Tabla 5.50: Requisito de información RI-11: Módulo.

ID	RI-12
Nombre	Preguntas
Descripción	Registra las interacciones de los estudiantes en forma de preguntas y respuestas dentro de la plataforma, lo que permite un seguimiento de las dudas y facilita la asistencia por parte del cuerpo docente o de otros estudiantes.
Contenido	Incluye la identificación del estudiante que realiza la pregunta, el texto de la pregunta, la respuesta proporcionada, fechas de realización y respuesta, y cualquier archivo adjunto relevante para la discusión.

Tabla 5.51: Requisito de información RI-12: Preguntas.

ID	RI-13
Nombre	Requisitos completados por el usuario
Descripción	Contabiliza los requisitos que cada usuario ha completado en los distintos módulos del sistema. Esto permite al sistema y a los instructores monitorear el progreso y asegurar que los estudiantes cumplan con los objetivos educativos establecidos.
Contenido	Incluye el identificador del usuario, el requisito completado, el módulo asociado y la fecha de finalización, permitiendo así una gestión efectiva del avance de cada estudiante.

Tabla 5.52: Requisito de información RI-13: Requisitos completados por el usuario.

ID	RI-14
Nombre	Requisitos de teoría
Descripción	Asocia los contenidos teóricos con los requisitos educativos específicos, estableciendo las dependencias que los estudiantes deben cumplir para progresar en el aprendizaje teórico del curso.
Contenido	Relaciones entre teorías y requisitos que deben ser completados para acceder a dichos contenidos, asegurando una secuencia lógica de aprendizaje.

Tabla 5.53: Requisito de información RI-14: Requisitos de teoría.

ID	RI-15
Nombre	Requisitos de ejercicios
Descripción	Define las dependencias entre los ejercicios prácticos y los requisitos educativos, indicando qué conocimientos previos son necesarios para abordar cada ejercicio.
Contenido	Mapeo de los ejercicios con los requisitos correspondientes, lo que permite al sistema validar la preparación del estudiante antes de intentar resolver un ejercicio.

Tabla 5.54: Requisito de información RI-15: Requisitos de los ejercicios.

Capítulo 6

Diseño e Implementación

6.1. Diseño Pedagógico

El diseño pedagógico se centra en el aprendizaje basado en problemas [12]. Concretamente, se introduce la teoría relevante, que puede incluir explicaciones de texto, imágenes ilustrativas, y en ocasiones, material en formato de video. Una vez que el usuario ha absorbido el contenido teórico, se le presentan problemas prácticos relacionados para resolver.

Existen dos categorías principales de ejercicios, cada una con un peso distinto en la estructura de aprendizaje:

1. Ejercicios estándar: Son problemas prácticos que refuerzan la comprensión del usuario sobre el tema.

2. Ejercicios clave (*key_ exercise*): Estos ejercicios desempeñan un papel crucial en la progresión del usuario. Son indicativos de si el usuario ha comprendido los requisitos fundamentales del tema en cuestión. Se deben completar estos ejercicios para avanzar al siguiente tema o módulo. Si el usuario no logra superar estos ejercicios en un primer intento, se le presentan ejercicios adicionales para reforzar su comprensión antes de volver a intentarlo.

La plataforma corrige automáticamente para los módulos de C++, Java y Python, tal como se mencionó con anterioridad. Sin embargo, para el módulo de HTML+CSS+JS, se requiere una revisión manual por parte de un instructor. Esta revisión no solo califica el ejercicio, sino que también proporciona comentarios constructivos para guiar al usuario hacia la solución correcta.

Además de la corrección, la plataforma evalúa la limpieza y estructura del código presentado, ya que es esencial para un buen aprendizaje. Por lo tanto, se proporciona una retroalimentación detallada al estudiante no solo sobre la corrección de su respuesta, sino también sobre cómo mejorar y optimizar su código.

Finalmente, la plataforma tiene la capacidad de compilar ejercicios, permitiendo la verificación y corrección del código antes de su envío.

6.2. Arquitectura del Sistema

La interacción del usuario en el Frontend se lleva a cabo utilizando tecnologías estándar como HTML sin emplear *frameworks* adicionales para su desarrollo. Además, la interfaz es *Web Responsive*, lo que garantiza su adaptación a variados dispositivos. Desde el punto de vista de la seguridad, se han implementado medidas como la validación de entradas, previniendo así amenazas como los ataques XSS. Los datos de sesión se conservan en el cliente mediante cookies.

Por otro lado, el Backend se ha desarrollado con Flask. Se encarga de administrar las API RESTful y de establecer una conexión ininterrumpida con la base de datos PostgreSQL. La biblioteca *flask_ login* respalda las funcionalidades de autenticación y autorización, aportando una capa de seguridad adicional. La lógica de negocio del sistema reside principalmente en el Sistema de Tutoría Inteligente (ITS). Este subsistema es el encargado de la elección y evaluación de ejercicios, ofreciendo retroalimentación y monitoreando el avance del estudiante.

En relación con la seguridad del Backend, se han establecido roles y restricciones de acceso coherentes con las reglas de negocio previamente determinadas. Aunque la arquitectura actual no posee herramientas concretas orientadas a la escalabilidad, está concebida para ser resistente y estable.

Es esencial destacar que, la estructura general del sistema sigue el patrón Modelo-Vista-Controlador (MVC). La Vista reside en el Frontend, y el Controlador y Modelo en el Backend 6.1. Este patrón facilita la distinción entre la lógica de la interfaz de usuario y las operaciones, simplificando su mantenimiento.

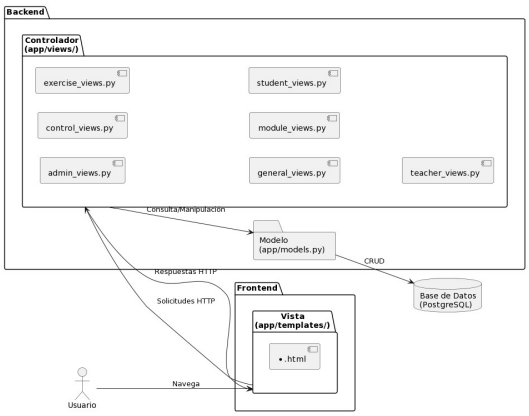

Figura 6.1: Arquitectura de sistema

6.3. Diseño de Base de Datos

La arquitectura de nuestra base de datos es el resultado de un exhaustivo análisis y diseño. Su estructura no solo atiende a las necesidades actuales, sino que también se anticipa a los retos futuros. A continuación, detallamos algunos de los elementos clave y las consideraciones adoptadas en su construcción:

1. **Tablas principales y su significado**: Las tablas como *users*, *modules*, *exercises*, *questions* y *theory* son la columna vertebral de la base de datos. En ellas, se almacena la información medular que alimenta las funcionalidades centrales del sistema. Estas tablas, además de contener datos primordiales, están interconectadas a través de relaciones específicas. Garantizando, así, la coherencia y la integridad de la información en todo momento.

2. **La importancia de los índices**: Los índices son herramientas fundamentales para acelerar las consultas y garantizar un rendimiento óptimo. Dentro de la base de datos, su uso es estratégico. Existen índices únicos, como el *users_email_key*, que aseguran la singularidad de ciertos registros. Por ejemplo, en el caso del correo electrónico, se garantiza que no existan duplicados. Los índices compuestos, como *student_modules_student_id_module_id_key*, son esenciales para optimizar las consultas que abarcan múltiples campos. Las tablas intermedias, que representan relaciones entre otras, también hacen uso de índices. En este caso, *exerciserequirements* y *theoryrequirements*. Estos índices, principalmente compuestos, agilizan y refuerzan las consultas relacionales.

3. **La normalización como estándar**: Uno de los principios fundamentales que se ha seguido es la normalización. Al estructurar la información de esta manera, se minimiza la redundancia y maximiza la eficiencia.

4. **Relaciones bien definidas**: La claridad en las relaciones entre tablas es esencial. Gracias a las claves primarias y foráneas, se ha conseguido una red de relaciones que no solo facilita la integración y consulta de datos sino que también refuerza la integridad de los mismos.

5. **Mirando hacia el futuro**: Más allá de las necesidades actuales, el diseño busca ser resiliente y adaptable. Pretendiendo que cualquier adición futura, ya sea en funcionalidades o módulos, se integre sin grandes complicaciones.

En resumen, la base de datos del sistema es robusta, detallada y diseñada para crecer. Teniendo como aspiración que, más allá de ser un simple almacén de datos, sea una herramienta eficiente que potencie la experiencia del usuario y facilite el trabajo de la ITS.

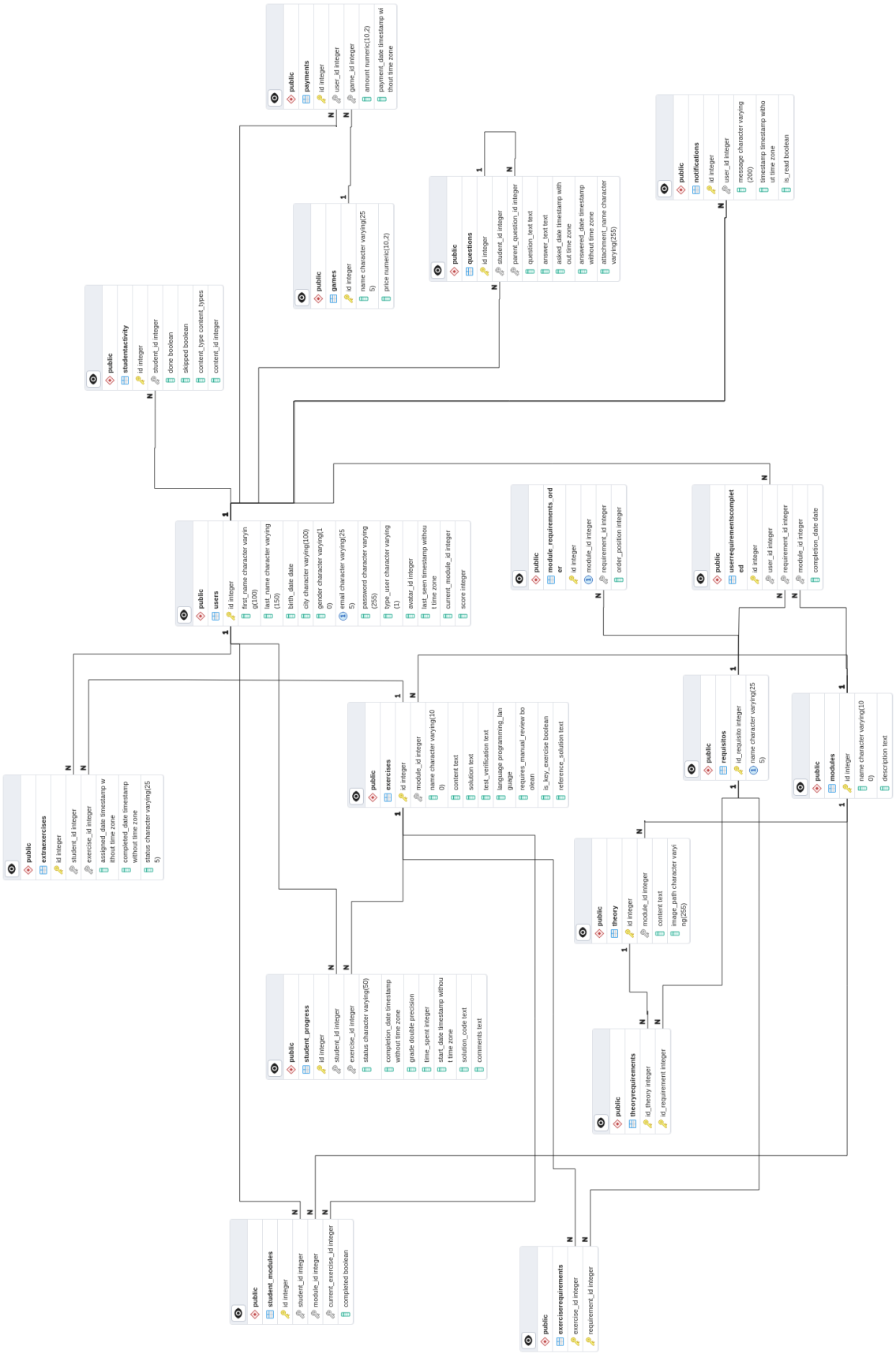

Figura 6.2: Diagrama del modelo de datos

6.4. Interfaz de Usuario

6.4.1. *Mockups* de la página web

Este apartado incluye una serie de *mockups* que representan las diferentes interfaces de usuario del sistema. Los *mockups* han sido diseñados utilizando la herramienta en línea *wireframe.cc* [63]. Esta herramienta permite la creación rápida y eficiente de prototipos de interfaces, facilitando la visualización y el diseño preliminar de las interacciones del usuario que se muestran a continuación.

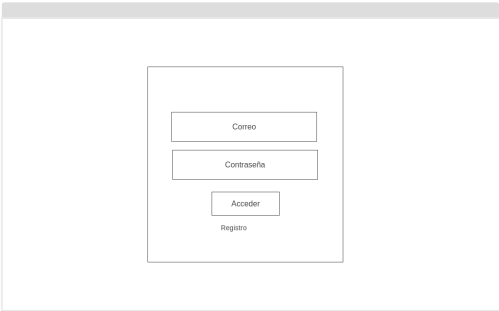

Figura 6.3: Inicio de Sesión para cualquier usuario.

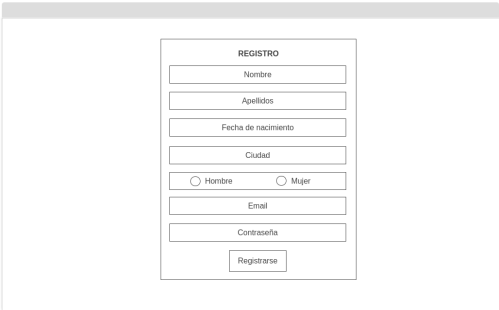

Figura 6.4: Registro para cualquier usuario.

Figura 6.5: Página Principal de la interfaz del estudiante.

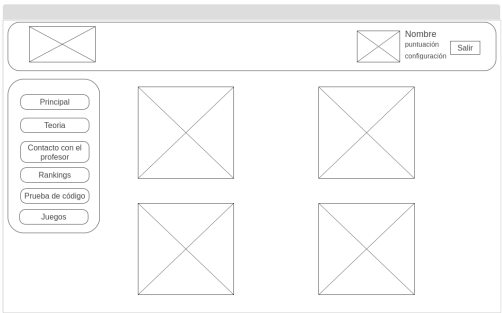

Figura 6.6: Apartado de Juegos de la interfaz del alumno.

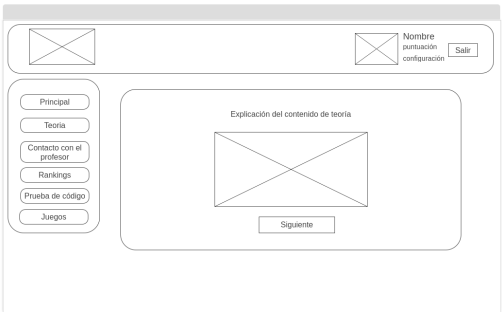

Figura 6.7: Visualización de la teoría para el estudiante.

Figura 6.8: Apartado de contacto con el profesor.

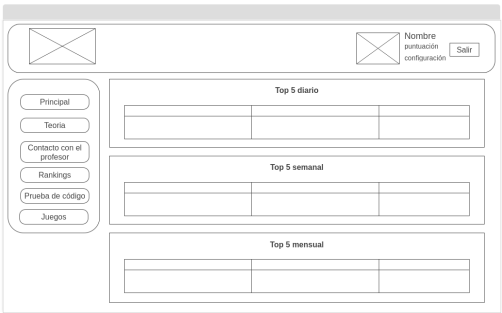

Figura 6.9: Apartado de Ranking de la interfaz del usuario.

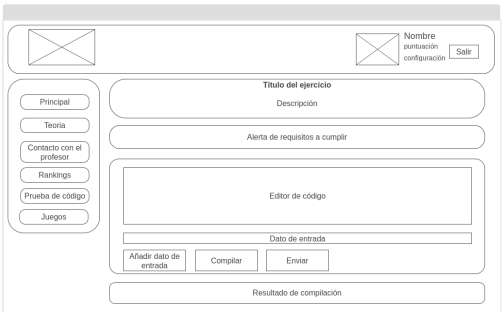

Figura 6.10: Visualización de los ejercicios para el usuario.

Figura 6.11: Apartado donde los estudiantes pueden revisar conceptos teóricos.

Figura 6.12: Apartado de configuración del estudiante.

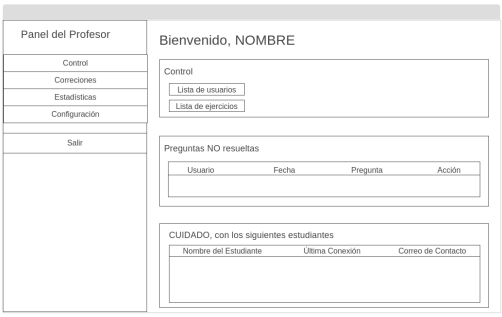

Figura 6.13: Dashboard principal del profesor.

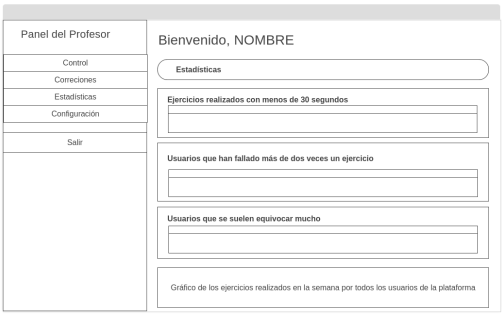

Figura 6.14: Pantalla de Estadísticas del profesor.

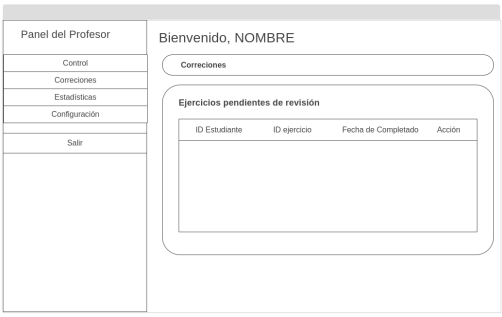

Figura 6.15: Pantalla con la lista de corrección de ejercicios.

Figura 6.16: Panel para facilitar la corrección los ejercicios para el profesor.

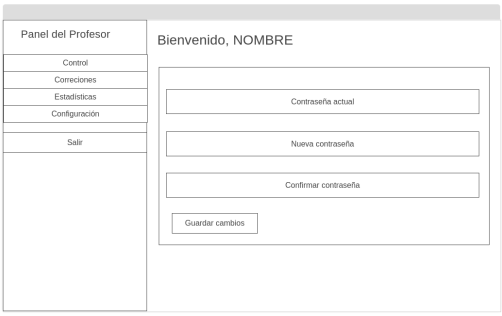

Figura 6.17: Apartado de Configuración del profesor.

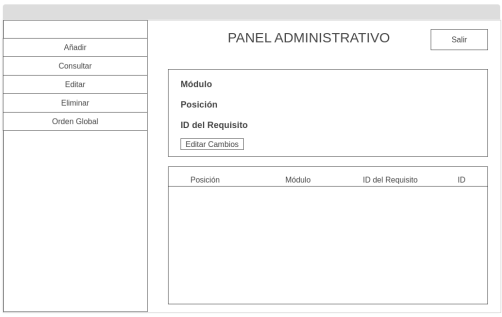

Figura 6.18: Visión general del panel administrativo.

6.4.2. Interacción Usuario-Sistema

El sistema está diseñado para una comunicación efectiva y amigable con el usuario. La fluidez en la navegación y la clara representación de opciones forman una experiencia de usuario destacable.

1. **Inicio de Sesión**: Al entrar, la primera pantalla es la de inicio de sesión. Aquí, el usuario introduce su nombre y contraseña. Si las credenciales son correctas, se redirige a la página principal. De lo contrario, un breve mensaje de error informa del problema.

2. **Registro**: Para nuevos usuarios, hay una pantalla especial donde se pide la información básica.

3. **Página Principal**: Una vez dentro, la página principal se despliega. Ofrece múltiples funciones: desde juegos hasta rankings. Iconos y etiquetas guían al usuario, haciendo la navegación intuitiva.

4. **Juegos, Teoría y Ejercicios**: Estas secciones son el corazón educativo del sistema. Los juegos son la motivación del estudiante. La teoría ofrece contenido educativo valioso y los ejercicios proponen retos prácticos que evalúan el aprendizaje del usuario.

5. **Contacto con el profesor y Ranking**: Dos secciones cruciales. En 'Contacto con el profesor', el usuario puede realizar consultas y ver sus anteriores dudas conjuntamente con los comentarios de algunos ejercicios. 'Ranking' motiva a los usuarios mostrando una lista de los más destacados.

6. **Interfaz del Profesor**: No solo los estudiantes tienen espacio en el sistema. Los profesores tienen su propia interfaz. Pueden observar estadísticas, corregir tareas y más. También tienen acceso a un panel administrativo.

7. **Notificaciones**: Finalmente, las notificaciones juegan un papel vital. Informan al usuario sobre correcciones, nuevos contenidos o interacciones. Esta comunicación directa asegura que el usuario esté siempre al tanto de los eventos relevantes.

El **Panel del Administrador** es una herramienta esencial para la gestión de la base de datos. Concretamente, ofrece una interfaz gráfica que evita la necesidad de interactuar a través de la terminal. Este enfoque agiliza las tareas cotidianas, y reduce la posibilidad de errores humanos que podrían surgir al gestionar datos de forma manual. Es importante mencionar que solo hemos mostrado un contexto general de la interfaz dentro de los mockups presentados. Estas representaciones cubren las funciones básicas, por ello, incluir más detalles sería redundante, ya que las operaciones fundamentales están claramente ilustradas.

En resumen, el sistema busca una interacción amena y productiva proporcionando las herramientas que necesitan los diferentes usuarios, presentadas de manera clara y accesible.

6.5. Implementación de Algoritmos

6.5.1. Corrección y evaluación de calidad del código

El verdadero reto comienza tras la entrega de un ejercicio por parte del estudiante. La lógica de corrección evalúa el código en términos de precisión y calidad. Si un código no alcanza un estándar mínimo, el sistema propone ejercicios adicionales, reforzando así las áreas de mejora del estudiante 6.19. La justificación detrás de esta lógica se centra en la premisa de que la repetición y el refuerzo son esenciales para consolidar el aprendizaje. Además, al establecer

estándares de calidad, se fomentan las buenas prácticas de programación desde las etapas iniciales de formación.

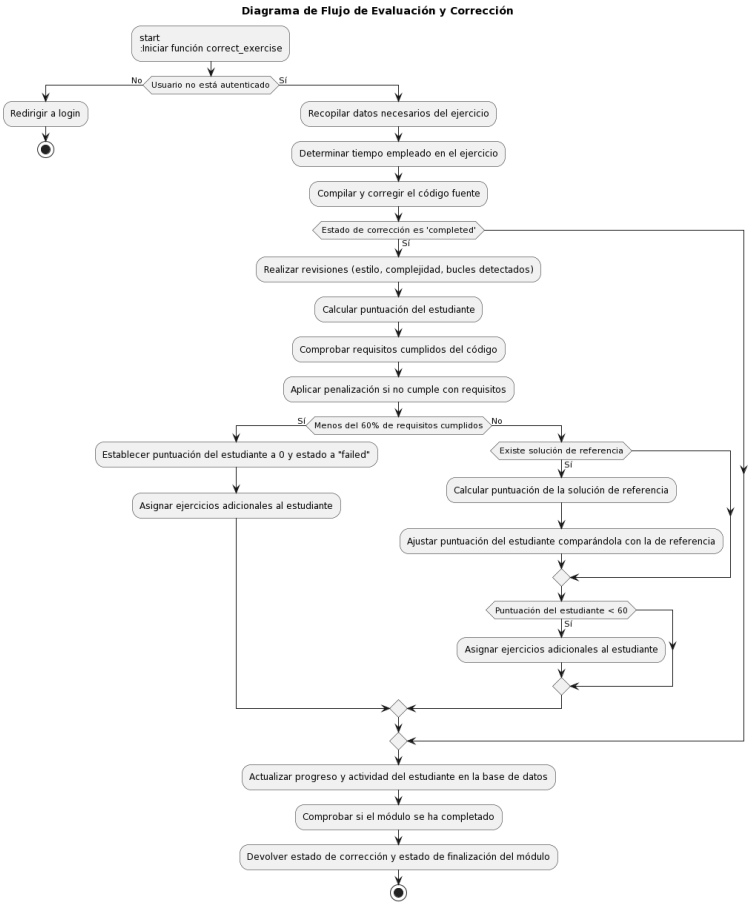

Figura 6.19: Diagrama de flujo para la selección de ejercicios

6.5.2. Selección y evaluación de ejercicios

El sistema prioriza la progresión estructurada del estudiante. Al ingresar, la plataforma identifica ejercicios en curso o determina el próximo paso en función de los requisitos del módulo. Esta estructura garantiza que, antes de enfrentarse a cualquier tarea, los conceptos teóricos relevantes sean presentados al estudiante 6.20. Esta estrategia se fundamenta en la pedagogía moderna, que sugiere que la teoría y la práctica deben ir de la mano para un aprendizaje óptimo.

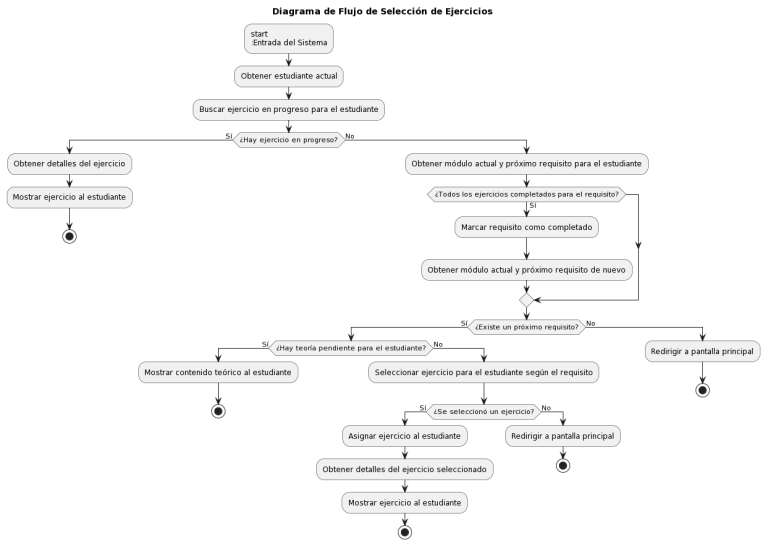

Figura 6.20: Diagrama de flujo para la selección de ejercicios

6.5.3. Cálculo de la puntuación

El sistema utiliza la función `calculate_score` para calcular la puntuación de un ejercicio. Esta función combina múltiples criterios que inciden en la calidad y eficacia del código del estudiante.

- **Problemas de Estilo**: Detectados mediante un análisis estático. Cada inconveniente identificado resta 5 puntos de los 100 iniciales.
- **Complejidad Ciclomática**: Un valor elevado (superior a 10) en la complejidad ciclomática suele indicar que el código puede ser difícil de mantener o probar. Por ello, se aplica una penalización significativa de 20 puntos. Esta cantidad ha sido establecida para subrayar la importancia de escribir código sencillo y fácilmente comprensible.
- **Bucles Detectados**: La presencia de bucles no es necesariamente un problema, pero puede llevar a un código menos eficiente o más complejo si no se usan adecuadamente. Se resta una cantidad menor, 15 puntos, con tal de incentivar el uso cuidadoso de estas estructuras.

La fórmula que resume todo el proceso es la siguiente:

$$\text{máx}\left(0, 100 - 5 \times \text{Problemas de Estilo} - 20 \times I_{\text{complejidad } >10} - 15 \times I_{\text{bucles detectados}}\right) \tag{6.1}$$

Cabe destacar que $I_{\text{condición}}$ actúa como una función indicadora. Toma el valor de 1 si la condición se cumple y 0 en caso contrario.

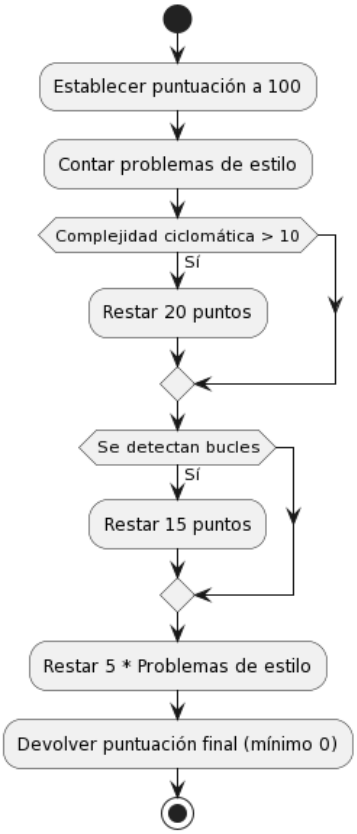

Figura 6.21: Diagrama de flujo que ilustra cómo se calcula la puntuación

Finalmente, la función tiene un control para que la puntuación no sea negativa, con tal de evitar la desmotivación del estudiante. Devuelve el máximo entre el valor calculado y cero.

6.5.4. Comprobaciones del código

Para evaluar el cumplimiento de ciertos requisitos educativos en el código, se emplea la técnica de Análisis de Árbol de Sintaxis Abstracta (AST) [46]. En este contexto, se utiliza una clase especializada denominada `RequirementVisitor`, que hereda de la clase base `ast.NodeVisitor`.

La clase `RequirementVisitor` es responsable de recorrer el AST generado a partir del código fuente. En su recorrido, identifica diferentes nodos que corresponden a constructos programáticos específicos: operadores básicos, estructuras de control, funciones, clases, entre otros.

Una vez que se encuentra un nodo de interés, la clase cambia el estado de una bandera correspondiente en un diccionario llamado *requirements_found* a True. Este diccionario actúa como un registro para los diferentes requisitos que el código cumple o no cumple.

Este mecanismo proporciona una manera eficiente y precisa de evaluar qué conceptos de programación se han aplicado en un determinado fragmento de código, lo que a su vez permite una evaluación pedagógica más informada.

Concretamente, dentro del constructor de la clase, se inicializa el diccionario con tal de tener un registro de los distintos elementos sintácticos y estructurales detectados en el código. Con el método `visit_BinOp`, se comprueba la presencia de operadores básicos como suma, resta, multiplicación y división. Usando `visit_BoolOp` se identifica el uso de operadores lógicos como AND y OR. El uso de las declaraciones `if-else` se detecta con el método `visit_If`. Los métodos `visit_While` y `visit_For` se encargan de identificar bucles `while` y `for`, respectivamente. Se detecta la definición de funciones mediante `visit_FunctionDef`. El método `visit_ClassDef` no solo detecta la definición de clases sino también la herencia mediante el atributo `bases`. Finalmente, `visit_List` y `visit_Dict` identifican el uso de listas y diccionarios, respectivamente.

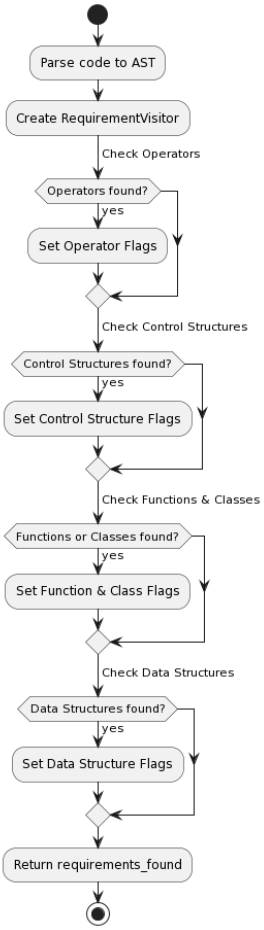

Figura 6.22: Diagrama de flujo que ilustra cómo se calcula la puntuación

6.5.5. Cálculo del número extra de ejercicios

El principal objetivo de la función `determine_number_of_extra_exercises` es adaptar de manera dinámica la carga de ejercicios asignados a un estudiante en función de su desempeño reciente, ofreciendo una experiencia de aprendizaje más personalizada.

Parámetros de Entrada

La función acepta los siguientes argumentos:

- **student_id**: Este identificador único del estudiante se utiliza para acceder a sus registros y recuperar su historial de rendimiento.
- **recent_num = 4**: Este parámetro establece que se tendrán en cuenta los últimos 4 ejercicios completados para evaluar la tasa de fracaso del estudiante. El número 4 se ha seleccionado como un equilibrio entre tener una muestra lo suficientemente grande para ser significativa, pero no tan grande como para ser desactualizada.
- **max_extra_exercises = 5**: Este es el número máximo de ejercicios adicionales que se pueden asignar. Limitar esto a 5 ayuda a prevenir una carga de trabajo abrumadora y repetitiva para el estudiante.
- **failure_rate_threshold = 0.35**: Se asignarán ejercicios adicionales si la tasa de fracaso supera este umbral. El valor de 0.35 se establece para permitir cierto margen de error pero todavía requiere una intervención si el rendimiento del estudiante es consistentemente bajo.

Flujo de Trabajo y Razonamiento

El flujo de trabajo de la función se puede desglosar en las siguientes etapas:

1. **Consulta de Requisitos**: La función primero identifica el último "requisito" completado por el estudiante para determinar desde dónde empezar.
2. **Búsqueda de Ejercicios Relacionados**: Se recuperan los IDs de los ejercicios asociados con el requisito actual para tener un conjunto de datos sobre el cual operar.
3. **Recolección y Análisis de Datos**: A continuación, se evalúa el rendimiento del estudiante en estos ejercicios, especialmente en términos de fracasos y éxitos, para calcular una tasa de fracaso.

4. **Normalización del Tiempo Empleado**: Los tiempos empleados en los ejercicios se normalizan para darles una escala uniforme, lo que permite una mejor comparación.
5. **Ponderación y Cálculo Final**: Los dos factores (tasa de fracaso y tiempo normalizado) se combinan en una suma ponderada para determinar el número final de ejercicios extra.

La fórmula utilizada para calcular el número de ejercicios extra es la siguiente:

$$N = \text{máx}\,(0, \text{round}\,(w_f \times f + w_t \times t) \times M) \tag{6.2}$$

Donde:

- $w_f = 0.7$ y $w_t = 0.3$ son los pesos asignados a la tasa de fracaso y al tiempo, respectivamente. Estos pesos reflejan la importancia relativa de cada factor; en este caso, la tasa de fracaso se considera más crítica.
- f es la tasa de fracaso actual del estudiante.
- t es el tiempo normalizado del último ejercicio.
- M es el número máximo de ejercicios extra permitidos.

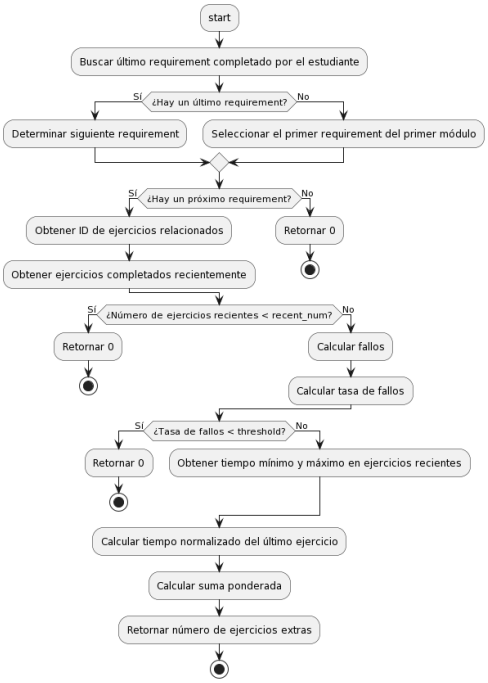

Figura 6.23: Diagrama de flujo que ilustra el cálculo del número extra de ejercicios

6.6. Implementación de Módulos de Aprendizaje Adaptativo

El núcleo de nuestro sistema educativo se centra en un modelo de aprendizaje adaptativo, diseñado para responder de manera dinámica al progreso y las necesidades del estudiante. La idea es no solo adaptar el tipo de material de aprendizaje presentado, sino también ajustar la cantidad y complejidad de los ejercicios asignados, todo en tiempo real.

La adaptación del sistema se logra mediante una combinación de técnicas de análisis de datos. En primer lugar, se recopilan métricas clave del rendimiento del estudiante, como las tasas de éxito y fracaso en ejercicios, el tiempo empleado. Estos datos se almacenan y analizan para determinar patrones y tendencias que luego son utilizadas para tomar decisiones adaptativas.

En el nivel más básico, el sistema cuenta con un conjunto predefinido de requisitos de aprendizaje, que son objetivos educativos específicos que el estudiante debe alcanzar. A medida que el estudiante avanza, el sistema evalúa continuamente su rendimiento y ajusta los requisitos de aprendizaje en función de este. Por ejemplo, si un estudiante supera con facilidad los ejercicios iniciales en un tema, el sistema puede decidir si avanzarle a un nuevo tema más rápidamente.

Además, en situaciones donde un estudiante demuestra dificultades persistentes, como una tasa de fracaso elevada en ejercicios recientes, el sistema asigna automáticamente ejercicios adicionales de refuerzo con tal ayudar al estudiante a mejorar en las áreas problemáticas, como se detalla en la sección anterior sobre el cálculo del número extra de ejercicios.

Un aspecto crítico que distingue nuestra implementación es la adaptabilidad basada en una solución de referencia proporcionada por el profesor. Este enfoque mitiga cualquier rigidez en el proceso de evaluación y garantiza que la retroalimentación sea contextual y ajustada.

El ajuste de la puntuación del estudiante se realiza mediante las siguientes fórmulas para calcular el fitness y la puntuación respectivamente:

$$\text{máx}\left(0,1 - \frac{|\text{teacher_score} - \text{student_score}|}{100}\right) \tag{6.3}$$

$$\text{student_score} \times = \text{fitness} \tag{6.4}$$

La variable `fitness` calcula la discrepancia entre la puntuación del estudiante y la del profesor. Este valor luego multiplica la puntuación original del estudiante para ajustarla.

En resumen, la implementación de módulos de aprendizaje adaptativo en nuestro sistema se lleva a cabo a través de un análisis cuidadoso de los datos del rendimiento del estudiante, seguido de la toma de decisiones algorítmicas para personalizar la experiencia de aprendizaje. Esta aproximación permite

no solo una mayor eficiencia en el proceso educativo, sino también una experiencia más personalizada y, por ende, más atractiva para el estudiante.

Capítulo 7

Despliegue

En este capítulo se va a abordar el despliegue de la aplicación Flask en el servidor de la Universidad de Granada *bahia.ugr.es* [20]. Concretamente, el despliegue se llevó a cabo mediante Podman [26] para la gestión de los contenedores. A continuación, se detallará el proceso de creación y configuración de las imágenes del contenedor, así como la metodología empleada.

7.1. Creación y Configuración de las Imágenes

Las imágenes del contenedor creadas fueron:

- **Imagen de la aplicación**: Se creó una imagen personalizada para la aplicación Flask, etiquetada como `nooocaaaa/programmingclasses:latest` C.2. Esta imagen incorpora todas las dependencias necesarias para la ejecución de la aplicación, así como las configuraciones específicas requeridas. La imagen fue construida utilizando Docker y posteriormente subida a Docker Hub [6], facilitando su distribución y posterior acceso.
- **Imagen de la base de datos**: De forma análoga, se generó una imagen personalizada para el servidor de base de datos PostgreSQL, etiquetada como `nooocaaaa/postgres-tfg:latest` C.3. Esta imagen está optimizada para integrarse con la aplicación Flask y también está disponible en Docker Hub [7].

7.2. Utilización de Podman

A partir de un archivo `docker-compose.yml` C.1, testeado en *localhost*, se adaptó la configuración para su uso con Podman. De esta manera, se aprovechó la compatibilidad y las ventajas de seguridad de esta herramienta, especialmente en entornos donde no se tienen privilegios de root.

En la configuración del fichero `podman-compose.yml` C.4, se establecieron dos servicios principales: el servicio de la aplicación (flask) y el servicio de la

base de datos (db). Cada uno de estos servicios se configuró cuidadosamente para garantizar una integración y operación eficientes.

Esta configuración facilitó un despliegue coherente y funcional de la aplicación, proporcionando un marco de trabajo estable y confiable para el funcionamiento de la aplicación Flask y su interacción con la base de datos PostgreSQL. Concretamente, la aplicación es accesible a través del siguiente enlace: http://bahia.ugr.es:35701/.

7.2.1. Contenedor flask_TFG

El servicio Flask se configuró con el nombre de contenedor `flask_TFG`, utilizando la imagen `nooocaaaa/programmingclasses:latest` anteriormente mencionada. Esta imagen, especialmente diseñada para la aplicación, contiene todas las dependencias y configuraciones necesarias.

Se mapeó el puerto `35701` del host al contenedor, proporcionando un punto de acceso a la aplicación para la comunicación con el exterior. Las variables de entorno se establecieron para configurar la conexión con la base de datos, definir claves secretas y especificar rutas de directorios críticos. Además, se definió una dependencia con el servicio de base de datos para asegurar que la base de datos estuviera disponible antes del inicio de la aplicación Flask. Es más, el comando de ejecución para el contenedor se configuró para iniciar la aplicación Flask.

7.2.2. Contenedor db

Por otro lado, el servicio de base de datos, etiquetado como `db`, utilizó la imagen `nooocaaaa/postgres-tfg:latest`, optimizada para trabajar en conjunto con la aplicación Flask.

Se expuso el puerto `5432` para permitir la comunicación con la base de datos. Las variables de entorno se configuraron para establecer los parámetros de la base de datos, incluyendo el nombre de la base de datos, el usuario y la contraseña. Para garantizar la persistencia de los datos, se utilizó un volumen, con tal que los datos se mantengan seguros y accesibles incluso después de reiniciar o detener el contenedor.

Capítulo 8

Pruebas y Resultados

Con el fin de evaluar exhaustivamente el sistema implementado, se realizaron pruebas con un pequeño pero significativo grupo de niños, compuesto por 3 participantes de distintas edades. Concretamente de 8, 12 y 15 años. Cada uno de ellos interactuó con el sistema de manera individual, en sesiones separadas. De esta manera, se obtuvo una evaluación más personalizada, detallada y precisa de la eficacia del sistema.

Los indicadores cuantitativos mostraron resultados bastante alentadores. Los jóvenes usuarios lograron completar los primeros ejercicios con éxito. Además, el tiempo que pasaron interactuando con la plataforma se mantuvo en un rango óptimo. Por lo tanto, se puede decir que el sistema fue eficaz en adaptarse a dificultad. Sin embargo, se observó que el tiempo óptimo de interacción con el sistema debería ajustarse según la edad del usuario, dado que las capacidades de atención y ritmo de lectura varían considerablemente en función de la edad.

En términos de usabilidad, las respuestas fueron positivas. La plataforma fue calificada como fácil de usar y práctica, lo cual indica que el diseño de la interfaz es adecuado. La claridad en las instrucciones también fue un punto destacado aunque a veces desconcertante para los más pequeños.

Entre las sugerencias aportadas, la más recurrente fue la adición de elementos como juegos con tal de incrementar la motivación y el compromiso. Este tipo de retroalimentación fue especialmente valiosa y se consideró como una mejora importante a realizar en el sistema. Además, la metodología de pruebas individuales permitió recoger opiniones y sugerencias sin la influencia del grupo.

En conclusión, las pruebas iniciales sugirieron que el sistema era efectivo en su misión de ofrecer una experiencia de aprendizaje adaptativo. Los comentarios y observaciones de los jóvenes participantes ofrecieron un conjunto invaluable de datos que sirvieron para optimizar y enriquecer el sistema.

Capítulo 9

Conclusiones y Trabajo Futuro

Este último capítulo sirve como el cierre al Trabajo de Fin de Grado presentado. En él, se busca sintetizar los hallazgos más significativos, destacando cómo estos contribuyen al campo de la enseñanza de la programación a niños. Asimismo, se esbozarán las direcciones para el trabajo futuro, delineando oportunidades para la expansión. Finalmente, se ofrecerá una reflexión personal sobre el impacto del proyecto.

9.1. Conclusiones

Este TFG se ha centrado en un reto importante: la enseñanza de la programación a niños en el contexto de una sociedad digitalizada en rápida evolución. A través de un enfoque multidisciplinario **??**, combinando pedagogía, tecnología educativa y diseño de sistemas, se ha desarrollado una plataforma innovadora y adaptable, destacándose en el campo de la enseñanza.

Un logro clave ha sido el desarrollo del Sistema de Tutorización Inteligente, que supera las barreras de los métodos tradicionales al ofrecer un aprendizaje personalizado y adaptativo. Este sistema no solo mejora la calidad del aprendizaje, sino que también permite a los estudiantes progresar a su propio ritmo, manteniendo las buenas prácticas de programación.

La colaboración con Codelearn S.L. ha sido instrumental, fusionando teoría y práctica para evaluar y mejorar el ITS, y así dar los primeros pasos hacia un sistema educativo más eficaz y atractivo.

9.2. Trabajo Futuro

A pesar de los avances logrados, el camino por delante es largo. Futuras líneas de investigación y desarrollo incluyen:

- **Variedad de Recursos Didácticos**: Una mayor diversidad en el tipo de ejercicios y evaluaciones, cómo relleno de código o cuestionarios

sobre ciertos temas, entre otros ejemplos.

- **Automatización de la Corrección**: Conseguir automatizar la corrección para los ejercicios HTML y que no se recurra al profesor.
- **Sistema de Chat-Bot**: Sistema conversacional para una primera resolución de dudas comunes en un ejercicio, como una explicación más extensa o ejemplos de ejecución.
- **Inteligencia Artificial**: El uso de algoritmos de aprendizaje automático que optimizaría la adaptabilidad del sistema a las necesidades individuales del estudiante, debido a la gran cantidad de datos con la que entrenaría.
- **Feedback en Tiempo Real**: La implementación de análisis del código mientras el estudiante lo este escribiendo con tal de poder hacer ajustes y/o comentarios inmediatos.
- **Tests A/B**: Realización de pruebas para evaluar la eficacia de distintas técnicas pedagógicas y elementos de diseño.

9.3. Reflexión Personal

Desde la concepción inicial del proyecto hasta la implementación técnica y las pruebas finales, cada etapa ha contribuido a un desarrollo profesional.

Empezando con la colaboración con Codelearn S.L., la experiencia ha sido invaluable. Ver un entorno real me ha ofrecido perspectivas que el ámbito académico raramente puede proporcionar. He aprendido a abordar problemas desde un punto de vista más aplicado, equilibrando teoría y práctica. Fortaleciendo, así, mi comprensión sobre cómo llevar soluciones tecnológicas desde el tablero de dibujo hasta una implementación funcional y real.

En el aspecto académico, este proyecto ha servido como una especie de trampolín para relacionar la ingeniería de software, los algoritmos y la pedagogía. Ganando una visión más clara de cómo la mayoría de disciplinas se pueden fusionar para producir soluciones educativas efectivas.

El desarrollo ágil adoptado para el proyecto fue una lección valiosa en sí misma. Me enseñó la importancia de la adaptabilidad y la agilidad en un entorno tecnológico que nunca se detiene. La retroalimentación continua y los ajustes constantes fueron cruciales para navegar a través de los desafíos que inevitablemente surgieron.

Apéndice A

Recursos Informáticos

En este apéndice se detalla todos los recursos de hardware y software que se han usado durante la ejecución de este proyecto. Esta información es esencial para entender el entorno de desarrollo y ejecución, permitiendo así una mejor replicación o continuación del trabajo realizado.

A.1. Hardware

Para el desarrollo y ejecución de este proyecto, se ha utilizado el siguiente hardware:

- Ordenador portátil *Xiaomi TM1703*.
  - Arquitectura de 64 bits.
  - 8 GB de Memoria RAM.
  - Procesador *Intel(R) Core(TM) 5-8250U CPU @ 1.60GHz*.

A.2. Software

En cuanto al software, se han utilizado las siguientes herramientas y plataformas:

- **Sistema Operativo**: Ubuntu 20.04.6 LTS.
- **Contenedores**: Docker, versión 24.0.5.
- **IDE**: Visual Studio Code 1.83.1
- **Base de datos**: PostgreSQL 16.0
- **Lenguaje de programación**: Python 3.8.10
- **Procesador de textos**: LaTex 3.14159265-2.6-1.40.20

- **Diseño de E-R**: draw.io
- **Modelado de datos**: PostgreSQL 16.0
- **Diagrama de flujo y arquitectura**: PlantUML v1.2023.12
- **Mockups**: Wireframe.cc
- **Bibliotecas Python**:
  - Flask [53]
  - Flask-Login [9]
  - Flask-SQLAlchemy [51]
  - Flask-CORS [14]
  - Flask-Bcrypt [10]
  - python-dotenv [35]
  - datetime [21]
  - werkzeug.utils [54]
  - radon.complexity [36]
  - cpplint [56]
  - lizard [59]
  - javalang [57]
  - libclang [58]
  - psycopg2-binary [60]

A.3. Recursos de Red y Servidores

Además del hardware y software mencionados, se usó el servidor *bahia* de la Universidad de Granada.

- **Nombre del Servidor**: *bahia.ugr.es*
- **Sistema Operativo**: Linux (Fedora)
- **Acceso**: A través de SSH (Secure Shell)
- **Usuario**: ncarrasco
- **Uso**: Despliegue y pruebas.

Para el despliegue de la aplicación y la base de datos, se usaron contendores.

- **Herramienta de Contenedores**: Podman
- **Aplicación Desplegada**: Aplicación Flask
- **Base de Datos Desplegada**: PostgreSQL
- **Servidor de Despliegue**: *bahia.ugr.es*
- **Configuración de Contenedores**: Vólumenes

Apéndice B

Guion de las pruebas con niños

Este guion tiene como objetivo mostrar los pasos seguidos para la realización de las pruebas de la plataforma. Concretamente, se busca evaluar la usabilidad, el interés generado y la eficacia educativa del sistema.

B.1. Preparación

- Preparar un ambiente tranquilo y sin distracciones.
- Asegurarse de que la plataforma funcione correctamente.
- Tener a mano material para tomar notas.

B.2. Participantes

- Edades entre 7 y 18 años.
- Con y sin experiencia previa en programación.

B.3. Actividades

Introducción y Consentimiento

- Explicar el objetivo de la prueba.

Prueba de Usabilidad

- Navegar por la plataforma.
- Completar un ejercicio.
- Utilizar las herramientas de ayuda o tutoriales, si están disponibles.

Prueba de Interés

- Observar las reacciones de los niños mientras usan la plataforma.
- Preguntarles qué les gusta y qué no.

Prueba de Eficacia Educativa

- Evaluar el conocimiento adquirido con un pequeño cuestionario oral.

B.4. Feedback y Conclusiones

- Pedir a los niños que proporcionen comentarios.
- Analizar los resultados y planificar mejoras para la plataforma.

Apéndice C

Ficheros usados para el despliegue

Este apéndice contiene los ficheros clave utilizados para el despliegue de la aplicación, proporcionando una visión detallada de las configuraciones de Podman y Docker. Se incluyen tanto los `Dockerfiles` para la creación de las imágenes personalizadas de la aplicación y la base de datos, como los archivos `docker-compose.yml` y `podman-compose.yml`.

```
version: '3'
services:
flask:
    container_name: flask_TFG
    image: nooocaaaa/programmingclasses:latest
    restart: unless-stopped
    ports:
    - "35701:35701"
    volumes:
    - ./app:/app
    - ./run.py:/run.py
    environment:
    - SQLALCHEMY_DATABASE_URI=postgresql://noelia:
        ↪ nocavi12@192.168.1.129:5432/mydatabase
    - SQLALCHEMY_TRACK_MODIFICATIONS=False
    - SECRET_KEY=
        ↪ fe701b7f09e6ef2e015591155de65a8cf85b160e6a75490a
    - UPLOAD_FOLDER=/app/asked_questions
    - STATIC_FOLDER=/app/static
    - CLANG_LIB=/usr/lib/llvm-10/lib
    command: bash -c "python run.py"
```

Listing C.1: Docker-compose de prueba

```
# Usa una imagen base de Python
FROM python:3.8

# Instala las herramientas del sistema necesarias
RUN apt-get update && apt-get install -y \
    cppcheck \
    clang \
    checkstyle \
    cppcheck \
    clang-tools \
    llvm

# Establece un directorio de trabajo
WORKDIR /

# Copia el archivo de requerimientos primero para aprovechar la cache de Docker
COPY requirements.txt /

# Instala las dependencias
RUN pip install --no-cache-dir -r requirements.txt

# Copia el resto de tu codigo
# COPY ./app /app
# COPY ./run.py /
# COPY ./.env /

# # Expone el puerto que usa tu app
# EXPOSE 5000

# # El comando para arrancar la app
# CMD ["flask", "run"]
```

Listing C.2: Dockerfile para la creación de la imagen de la app

```
FROM postgres:latest

# Copiar el script de backup a /docker-entrypoint-initdb.d
COPY backup.sql /docker-entrypoint-initdb.d/

# El script de backup se ejecutara automaticamente al iniciar el contenedor
```

Listing C.3: Dockerfile para la creación de la imagen de la base de datos

```
version: '3'

services:
flask:
    container_name: flask_TFG
    image: nooocaaaa/programmingclasses:latest
    restart: unless-stopped
    ports:
    - "35701:35701"
    volumes:
    - ./app:/app
    - ./run.py:/run.py
    environment:
    - SQLALCHEMY_DATABASE_URI=postgresql://noelia:nocavi12@db
        ↪ :5432/mydatabase
    - SQLALCHEMY_TRACK_MODIFICATIONS=False
    - SECRET_KEY=
        ↪ fe701b7f09e6ef2e015591155de65a8cf85b160e6a75490a
    - UPLOAD_FOLDER=/app/asked_questions
    - STATIC_FOLDER=/app/static
    - CLANG_LIB=/usr/lib/llvm-10/lib
    depends_on:
    - db
    command: bash -c "python run.py"

db:
    image: nooocaaaa/postgres-tfg:latest
    restart: unless-stopped
    environment:
    POSTGRES_DB: mydatabase
    POSTGRES_USER: noelia
    POSTGRES_PASSWORD: nocavi12
    ports:
    - "5432:5432"
    volumes:
    - postgres_data:/var/lib/postgresql/data

volumes:
postgres_data:
```

Listing C.4: Fichero Podman-compose para desplegar la bbdd y la app en el servidor

Apéndice D

Manual de usuario

D.1. Captura de EScritorio

D.2. Capturas del Profesor

D.3. Capturas del Administrador

Glosario

ITS: Sistema Inteligente de Tutorización (Intelligent Tutoring System). Es un sistema que utiliza tecnologías de inteligencia artificial para proporcionar enseñanza personalizada y apoyo educativo a los estudiantes, adaptándose a las necesidades individuales de aprendizaje.

Hash y Salting: Técnicas de seguridad para el almacenamiento seguro de contraseñas. El "hashingçonvierte una contraseña en una cadena de caracteres única, mientras que "salting.ªñade datos aleatorios al hash para aumentar la seguridad.

HTTPS: Protocolo seguro para la transferencia de datos en la web. Utiliza el protocolo SSL/TLS para encriptar la información transmitida.

Flujo: Secuencia de pasos o actividades en un proceso o escenario. En el contexto de programación, se refiere a la ejecución ordenada de instrucciones en un programa.

Web Responsive: Diseño web adaptable a diferentes dispositivos. Utiliza hojas de estilo en cascada (CSS) para ajustar el diseño según el tamaño de la pantalla.

Gamificación: Uso de elementos de juego en entornos no lúdicos. Se emplea para aumentar la motivación y la participación en actividades que normalmente se considerarían tediosas.

Aprendizaje Basado en Problemas (ABP): Método pedagógico que utiliza problemas reales o simulados como punto de partida para el aprendizaje.

API RESTful: Interfaz de programación de aplicaciones que sigue los principios REST para la comunicación entre sistemas.

Backend: Parte del sistema que se encarga de la lógica de negocio, el acceso a la base de datos y la comunicación con diferentes servicios.

Clave Primaria: Campo único en una tabla de base de datos que se utiliza para identificar registros de forma única.

Clave Foránea: Campo en una tabla de base de datos que se utiliza para establecer una relación con otra tabla.

Normalización: Proceso de organización de una base de datos para reducir la redundancia y mejorar la integridad de los datos.

Análisis de Árbol de Sintaxis Abstracta (AST): Técnica de análisis de código que genera un árbol para representar su estructura sintáctica.

Complejidad Ciclomática: Métrica que mide la complejidad de un programa basada en el número de caminos independientes a través del código.

Función Indicadora: Función matemática que toma el valor de 1 si una condición específica se cumple y 0 en caso contrario.

Requisitos de Aprendizaje: Objetivos educativos específicos que un estudiante debe alcanzar en un sistema de aprendizaje adaptativo.

Tasa de Fracaso: Proporción de ejercicios o tareas que un estudiante no ha podido completar con éxito.

Ponderación: Asignación de importancia relativa a diferentes criterios o variables en un cálculo.

Análisis Estático: Examinar el código en busca de problemas sin ejecutar el programa.

Adaptabilidad: Capacidad de un sistema para ajustarse dinámicamente a las necesidades o comportamientos del usuario.

Fitness: Métrica que evalúa la calidad o eficacia de una solución en comparación con una solución de referencia.

Indicadores Cuantitativos: Métricas numéricas utilizadas para evaluar el rendimiento o eficacia de un sistema.

Usabilidad: Medida de la eficacia, eficiencia y satisfacción con la que los usuarios pueden realizar tareas en un sistema.

Metodología de Pruebas Individuales: Enfoque de evaluación que implica probar el sistema con cada usuario de forma separada para obtener retroalimentación más precisa.

Optimización: Proceso de hacer cambios en un sistema para mejorarlo en términos de eficiencia, eficacia o experiencia del usuario.

Bibliografía

[1] Plantulm. https://plantuml.com/es/, 2023. Accedido el: 29 de Octubre, 2023.

[2] V. Aleven and F. Xhakaj. Multidisciplinary adaptability in intelligent tutoring systems. *Educational Technology Research and Development*, 2022.

[3] Vincent Aleven et al. Help seeking and intelligent tutoring systems: Theoretical perspectives and a step towards theoretical integration. *Educational Psychology Review*, 2016.

[4] C. J. Butz, S. Hua, and R. B. Maguire. A web-based bayesian intelligent tutoring system for computer programming. Technical report, Department of Computer Science, University of Regina, 2006.

[5] Bernat Foj Capell. Codelearn. https://www.codelearn.es, 2013. Accedido el: 24 de Octubre, 2023.

[6] Noelai Carrasco Vilar. https://hub.docker.com/r/nooocaaaa/programmingclasses.

[7] Noelia Carrasco Vilar. https://hub.docker.com/r/nooocaaaa/postgres-tfg.

[8] A. Corbett and J. Anderson. Effectiveness of intelligent tutoring systems. *International Journal of Human-Computer Studies*, 2001.

[9] Max Countryman. Flask-login. https://flask-login.readthedocs.io/, 2012-2023. Accedido el: 31 de Octubre, 2023.

[10] Max Countryman. Flask-bcrypt. https://flask-bcrypt.readthedocs.io/, 2013-2023. Accedido el: 31 de Octubre, 2023.

[11] Tyne Crow, Andrew Luxton-Reilly, and Burkhard Wünsche. Intelligent tutoring systems for programming education: a systematic review. pages 53–62, 01 2018.

[12] Servicio de Innovación Educativa. Aprendizaje basado en problemas. *Guías rápidas sobre nuevas metodologías, Disponible en http://innovacioneducativa. upm. es/guias/Aprendizaje_ basado_ en_ problemas. pdf*, 2008.

[13] Robert B.K. Dewar and Edmond Schonberg. Computer science education: Where are the software engineers of tomorrow? *Open Forum*, 2001.

[14] Cory Dolphin. Flask-cors. `https://flask-cors.readthedocs.io/`, 2014-2023. Accedido el: 31 de Octubre, 2023.

[15] DragonBox. The dragonbox method. `https://dragonbox.com/`, 2022. Accedido el: 25 de Octubre, 2023.

[16] Paula J. Durlach and Anne M. Lesgold. Learner modeling in intelligent tutoring systems. *Educational Technology*, 2008.

[17] S. D'Mello and A. Graesser. Emotions and learning in intelligent tutoring systems. *Journal of Educational Psychology*, 2013.

[18] Discovery Education. Dreambox learning. `http://www.dreambox.com/`, 2022. Accedido el: 24 de Octubre, 2023.

[19] S. E. Fancsali and S. Nixon, T. andRitter. Data-driven methodologies in intelligent tutoring systems. *Journal of Learning Analyticsy*, 2019.

[20] Juan Manuel Fernández Luna and José Manuel Benítez Sánchez. `https://bahia.ugr.es`.

[21] Python Software Foundation. datetime. `https://docs.python.org/3/library/datetime.html`, 2001-2023. Accedido el: 31 de Octubre, 2023.

[22] Douglas Frye and Elliot Soloway. Interface design: A neglected issue in educational software. In *Proceedings of the SIGCHI/GI Conference on Human Factors in Computing Systems and Graphics Interface*, CHI '87, pages 93–97. Association for Computing Machinery, 1986.

[23] Inc. GlassLab y Take-Two Interactive Software. Civilization edu. `https://civilization.com/es-ES/`, 2016. Accedido el: 25 de Octubre, 2023.

[24] Google. Google it support professional certificate. `https://grow.google/it-cert/`, 2023. Accedido el: 25 de Octubre, 2023.

[25] Arthur C. Graesser et al. Autotutor: A tutor with dialogue in natural language. *Behavior Research Methods, Instruments, & Computers*, 2009.

[26] Red Hat. `https://podman.io/`.

[27] N. Heffernan and C. Heffernan. The assistments ecosystem: Building a platform that brings scientists and teachers together for minimally invasive research on human learning and teaching. *International Journal of Artificial Intelligence in Education*, 2014.

[28] N. Heffernan and C. Heffernan. Human-agent interaction in intelligent tutoring systems. *International Journal of Artificial Intelligence in Education*, 2019.

[29] IIS International. *Adaptative Learning in real world contexts*, Zagreb, 2016. 13ava Conferencia Internacional sobre los Sistemas de Tutorización Inteligente.

[30] IIS International. *Artifical Intelligence and Beyond: from Alpha to Omega*, Athens (Online Conference), 2020. 16ava Conferencia Internacional sobre los Sistemas de Tutorización Inteligente.

[31] IIS International. *Intelligent Tutoring Sytems in an Online world*, Athens (Online Conference), 2021. 17ava Conferencia Internacional sobre los Sistemas de Tutorización Inteligente.

[32] K. R. Koedinger, A. T. Corbett, and C. Perfetti. The knowledge-learning-instruction framework: Bridging the science-practice chasm to enhance robust student learning. *Cognitive Science*, 2013.

[33] Kenneth R. Koedinger et al. Intelligent tutoring goes to school in the big city. *International Journal of Artificial Intelligence in Education*, 1997.

[34] Andrew Ng Daphne Koller. Coursera for campus. `https://www.coursera.org/campus`, 2011. Accedido el: 25 de Octubre, 2023.

[35] Saurabh Kumar. python-dotenv. `https://github.com/theskumar/python-dotenv`, 2016-2023. Accedido el: 31 de Octubre, 2023.

[36] Michele Lacchia. Radon. `https://radon.readthedocs.io/`, 2014-2023. Accedido el: 31 de Octubre, 2023.

[37] J. A. Large and Jamshid Beheshti. Interface design, web portals, and children. *Library Trends*, 54(2):318–342, 2005.

[38] Carnegie Learning. Fast forword. `https://www.scilearn.com/#`, 2021. Accedido el: 24 de Octubre, 2023.

[39] Linda Liukas. The poetry of programming. TEDxCERN, 2015. `https://www.youtube.com/watch?v=-jRREn6ifEQ`.

[40] Lockheed Martin. F-35 lightning ii training system. https://www.lockheedmartin.com/en-us/products/f-35-lightning-ii-training-systems.html, 2021. Accedido el: 25 de Octubre, 2023.

[41] Medtronic. Touch surgery simulator. https://www.touchsurgery.com/simulations, 2019. Accedido el: 25 de Octubre, 2023.

[42] Tom Murray. Authoring intelligent tutoring systems. *Journal of Educational Computing Research*, 2003.

[43] H. S. Nwana. Intelligent tutoring systems: An overview. *Artificial Intelligence Review*, 1990.

[44] Hadi Partovi. Computer science is for everyone. TEDxRainier, 2014. https://www.youtube.com/watch?v=FpMNs7H24X0.

[45] Arnold Pears, Stephen Seidman, Lauri Malmi, Linda Mannila, Elizabeth Adams, Jens Bennedsen, Marie Devlin, and James Paterson. A survey of literature on the teaching of introductory programming. *ACM SIGCSE Bulletin*, 39(4):204–223, 2007.

[46] Python. Librería ast — Árboles de sintaxis abstracta. https://docs.python.org/es/3/library/ast.html, 2001-2023. Accedido el: 30 de Octubre, 2023.

[47] Graesser R. A., Hu A. C., and H. X. Holden. Design recommendations for intelligent tutoring systems. *Ethics and Information Technology*, 2017.

[48] J. Radianti and T. A. Majchrzak. Virtual and augmented reality in intelligent tutoring systems. *Journal of Educational Technology*, 2020.

[49] J. Reich and J. A. Ruipérez-Valiente. Equity and access in intelligent tutoring systems. *Journal of Learning Sciences*, 2019.

[50] Anthony Robins, Janet Rountree, and Nathan Rountree. Learning and teaching programming: A review and discussion. *Computer Science Education*, 13(2):137–172, 2003.

[51] Armin Ronacher. Flask-sqlalchemy. https://flask-sqlalchemy.palletsprojects.com/, 2011-2023. Accedido el: 31 de Octubre, 2023.

[52] Seguridad Social. Cotización y recaudación de trabajadores para el 2021. https://www.seg-social.es/wps/portal/wss/internet/Trabajadores/CotizacionRecaudacionTrabajadores/36537, 2023. Accedido el: 26 de Octubre, 2023.

[53] Flask Development Team. Flask. https://flask.palletsprojects.com/, 2010-2023. Accedido el: 31 de Octubre, 2023.

[54] Pallets Team. Werkzeug. https://werkzeug.palletsprojects.com/, 2010-2023. Accedido el: 31 de Octubre, 2023.

[55] Ugur Tevfik Kaplancali and Zafer Demirkol. Teaching coding to children: A methodological approach. *International Journal of Elementary Education*, 6(4):32–37, 2017.

[56] Unknown. cpplint. https://github.com/cpplint/cpplint, Unknown. Accedido el: 11 de Noviembre, 2023.

[57] Unknown. javalang. https://github.com/c2nes/javalang, Unknown. Accedido el: 11 de Noviembre, 2023.

[58] Unknown. libclang. https://clang.llvm.org/doxygen/group__CINDEX.html, Unknown. Accedido el: 11 de Noviembre, 2023.

[59] Unknown. Lizard. https://github.com/terryyin/lizard, Unknown. Accedido el: 11 de Noviembre, 2023.

[60] Unknown. psycopg2-binary. https://pypi.org/project/psycopg2-binary/, Unknown. Accedido el: 11 de Noviembre, 2023.

[61] Kurt VanLehn. The relative effectiveness of human tutoring, intelligent tutoring systems, and other tutoring systems. *Educational Psychologist*, 2011.

[62] Noelia Carrasco Vilar. Repositorio del proyecto. https://github.com/noocaaa/TFG_WEB/, 2023. Accedido el: 30 de Octubre, 2023.

[63] Wireframe.cc. Wireframe. http://www.wireframe.cc/, 2023. Accedido el: 30 de Octubre, 2023.

[64] Beverly Park Woolf. Building intelligent interactive tutors. *Morgan Kaufmann*, 2009.

[65] MIT y Harvard. edx. https://www.edx.org, 2012. Accedido el: 25 de Octubre, 2023.

[66] O. Zawacki-Richter and V. I. Marín. Ethics and privacy in intelligent tutoring systems. *International Journal of Educational Technology in Higher Education*, 2019.